Данное методическое пособие посвящено методам разработки настольных клиентских Windows-приложений с использованием языка C# и Windows Forms.

В основном при выполнении упражнений приложения создаются обучаемыми с нуля. Для изучения наиболее сложных вопросов используются готовые стартовые проекты. Исходные файлы для выполнения таких заданий находятся в сетевой папке (по указанию преподавателя). Скопируйте их на локальную машину перед выполнением упражнений.

Составитель: Осипов Н. А.

СОДЕРЖАНИЕ

Лабораторная работа 1. Введение в разработку форм 4
Упражнение 1. Настройка прямоугольной формы Windows 6
Упражнение 2. Создание непрямоугольной формы Windows 9
Упражнение 3. Создание наследуемой формы 10
Упражнение 4. Создание MDI-приложения 11
Упражнение 5 (контрольное). Создание пользовательской формы 14

Лабораторная работа 2. Работа с элементами управления 16
Упражнение 1. Обработка событий Click и MouseMove 16
Упражнение 2. Работа со списками 17
Упражнение 3. Создание и использование элемента управления ToolStrip 21
Упражнение 4. Использование элемента управления StatusStrip 23
Упражнение 5. Работа с контейнерными элементами управления 24
Упражнение 6. Реализация простого интерфейса ввода-вывода 26
Упражнение 7. Добавление и удаление элементов управления в режиме работы приложения 29
Упражнение 8. Проверка вводимых значений. События KeyPress и Validating. Элемент управления ErrorProvider ... 31
Упражнение 9 (контрольное). Реализация интерфейса 33

Лабораторная работа 3. Создание элементов управления 33
Упражнение 1. Создание составного элемента управления 33
Упражнение 2. Создание специализированного элемента управления 35
Упражнение 3. Создание расширенных элементов управления 36
Упражнение 4 (контрольное). Форма валидации данных 37

Лабораторная работа 4. Использование окон диалога 38
Упражнение 1. Использование стандартных окон диалога 38
Упражнение 2. Создание пользовательских модальных диалоговых окон 40
Упражнение 3. Взаимодействие компонентов формы и класса приложения 43
Упражнение 4. Создание немодальных окон диалога 46
Упражнение 5 (контрольное). Расчет функции 48

Лабораторная работа 5. Взаимодействие управляемого и неуправляемого кода 48
Упражнение 1. Получение данных с использованием библиотеки объектов Microsoft Excel 49
Упражнение 2. Использование COM-компонента для создания PDF-приложения 52

Упражнение 3. Вызов функции API 53
Упражнение 4. Программное создание документа Microsoft Word 55

Лабораторная работа 6. Графика и анимация в формах 58
Упражнение 1. Вывод графика функции 58
Упражнение 2. Реализация анимации 63
Упражнение 3. Использование компонента Chart 64

Лабораторная работа 7. Асинхронное программирование 66
Упражнение 1. Работа с компонентом BackgroundWorker 66
Упражнение 2. Использование делегатов 68
Упражнение 3. Асинхронный запуск произвольного метода 69
Упражнение 4. Асинхронное программирование с использованием ключевых слов async и await 72
Упражнение 5 (контрольное). Реализация алгоритма во вторичном потоке 73

Лабораторная работа 8. Повышение удобства использования приложений 74
Упражнение 1. Использование сериализации для сохранения коллекции объектов 74
Упражнение 2. Создание контекстной справки 77
Упражнение 3. Использование справочного файла 78
Упражнение 4. Добавление всплывающих подсказок 79
Упражнение 5. Автоматический выбор языка при запуске приложения 80
Упражнение 6. Локализация приложения 81

Лабораторная работа 9. Настройка приложений для распространения 83
Упражнение 1. Использование строго именованной сборки 83
Упражнение 2. Работа с глобальным кэшем сборок 84
Упражнение 3. Создание и использование файлов конфигурации приложения 86
Упражнение 4. Публикация приложения с помощью ClickOnce в сетевой папке 88

Итоговое контрольное задание 89

Литература 90

Лабораторная работа 1. Введение в разработку форм

Формы Windows — это основной компонент пользовательского интерфейса. Формы предоставляют контейнер, который содержит элементы управления, меню и позволяет отображать приложение в уже привычной и единообразной модели. Формы могут реагировать на события мыши и клавиатуры, поступающие от пользователя, и выводить на экран данные для пользователя с помощью элементов управления, которые содержатся в форме.

Формы Windows содержат множество свойств, позволяющих настраивать их внешний вид и поведение. Просматривать и изменять эти свойства можно в окне **Properties** конструктора при разработке, а также программно во время выполнения приложения.

В таблице перечислены некоторые свойства форм Windows, отвечающие за внешний вид и поведение приложения:

Свойство	**Описание**
Name	Задает имя классу **Form**, показанному в конструкторе. Данное свойство задается исключительно во время разработки
BackColor	Указывает цвет фона формы
Enabled	Указывает, может ли форма принимать ввод от пользователя. Если свойству **Enabled** задано значение **False**, все элементы управления формы также блокируются
ForeColor	Указывает цвет переднего плана формы, то есть цвет выводимого текста. Если отдельно не указать значение свойства **ForeColor** элементов управления формы, они примут то же значение
FormBorderStyle	Указывает вид и поведение границы и строки заголовка формы. Значения свойства:
	None - Форма не имеет границы, не может быть минимизирована или развернута до максимальных размеров и у нее нет экранной кнопки управления окном и кнопки справки
	FixedSingle - Форма имеет тонкую границу, и размеры формы нельзя изменить во время выполнения. Форма может быть минимизирована, развернута до максимальных размеров, и иметь кнопку справки или кнопку управления окном, что определяется остальными свойствами
	Fixed3D - Форма имеет объемную границу, и размеры формы нельзя изменить во время выполнения. Форма может быть минимизирована, развернута до максимальных размеров, и иметь кнопку справки или кнопку управления окном, что определяется остальными свойствами
	FixedDialog - Форма имеет тонкую границу, и размеры формы нельзя изменить во время выполнения. У формы нет экранной кнопки управления окном, но может быть кнопка справки, что

Свойство	Описание
	определяется остальными свойствами. Форму можно минимизировать и развернуть до максимальных размеров
	Sizable - Форма имеет настройки по умолчанию, но они могут изменяться пользователем. Форма может быть минимизирована, развернута до максимальных размеров, и иметь кнопку справки, что определяется остальными свойствами
	FixedToolWindow - Форма имеет тонкую границу, и размеры формы нельзя изменить во время выполнения. Форма содержит только кнопку закрытия
	SizableToolWindow - Форма имеет тонкую границу, и размеры формы могут быть изменены пользователем. Форма содержит только кнопку закрытия
Location	Когда свойству **StartPosition** задано значение **Manual**, это свойство указывает исходное положение формы относительно верхнего левого угла экрана
MaximizeBox	Указывает, есть ли у формы кнопка **MaximizeBox**
MaximumSize	Устанавливает максимальный размер формы. Если задать этому свойству размер 0; 0, у формы не будет верхнего ограничения размера
MinimizeBox	Указывает, есть ли у формы кнопка **MinimizeBox**
MinimumSize	Устанавливает минимальный размер формы, который пользователь может задать
Opacity	Устанавливает уровень непрозрачности или прозрачности формы от 0 до 100%. Форма, непрозрачность которой составляет 100%, полностью непрозрачна, а форма, имеющая 0 % непрозрачности, наоборот, полностью прозрачна
Size	Принимает и устанавливает исходный размер формы
StartPosition	Указывает положение формы в момент ее первого выведения на экран
Text	Указывает заголовок формы
TopMost	Указывает, всегда ли форма отображается поверх всех остальных форм, свойству **TopMost** которых не задано значение **True**
Visible	Указывает, видима ли форма во время работы
WindowState	Указывает, является ли форма минимизированной, развернутой до максимальных размеров, или же при первом появлении ей задан размер, указанный в свойстве **Size**

Выполнив упражнения этого модуля, вы научитесь настраивать форму Windows, создавать непрямоугольные формы, а также на основе базовой формы создавать наследуемые (производные) формы.

Упражнение 1. Настройка прямоугольной формы Windows

➢ **Создание нового проекта**

1. Откройте Visual Studio и создайте новый проект Windows Forms. Проект откроется с формой по умолчанию с именем **Form1** в конструкторе.

2. Выберите форму в конструкторе. Свойства формы отображаются в окне **Properties**.

3. В окне **Properties** задайте свойствам значения, как указано ниже:

Свойство	**Значение**
Text	Trey Research
FormBorderStyle	Fixed3D
StartPosition	Manual
Location	100; 200
Opacity	75%

4. Перетащите три кнопки из **Toolbox** в форму и разместите их так, как вам будет удобно.

5. Поочередно выберите каждую кнопку и в окне **Properties** задайте свойству кнопок **Text** значения **Border Style, Resize** и **Opacity.**

6. Для кнопки **Border Style** задайте свойство **Anchor** – **Top, Left.**

➢ **Реализация обработчиков событий**

7. В конструкторе дважды щелкните кнопку **Border Style**, чтобы открыть окно с кодом обработчика события **Buttonl Click.** Добавьте в этот метод следующую строку кода:

```
this.FormBorderStyle = FormBorderStyle.Sizable;
```

8. Возвратитесь в окно конструктора, дважды щелкните кнопку Resize и добавьте следующую строку:

```
this.Size = new Size(300, 500);
```

9. Возвратитесь в окно конструктора, дважды щелкните кнопку Opacity и добавьте следующую строку:

```
this.Opacity = 1;
```

➢ **Запуск готового решения**

10. Для построения решения выберите меню **Build (Построение)**, далее команду **Build Solution (Построить решение)**. При наличии ошибок исправьте их и снова постройте решение. В дальнейшем при необходимости выбора последовательности действий очередность команд будет описываться, например, так: **Build | Build Solution.**

11. Нажмите Ctrl + F5 или выберите **Debug (Отладка) | Start Without Debugging (Запуск без отладки)**, чтобы запустить приложение. Щелкайте каждую кнопку и наблюдайте, как изменяется вид формы.

12. Измените поочередно расположение левой и верхней границ формы и сравните поведение кнопок внутри формы. Обратите внимание, что расстояние до этих границ от кнопки **Border Style** остается постоянным. Почему?

➢ **Создание новой формы**

13.Добавьте новую форму: меню **Project (Проект)** | **Add Windows Form**…(Добавить форму Windows) далее укажите Windows Form.

14.В окне **Add New Item** (Добавление нового элемента) в поле **Name** укажите название формы: nForm.cs и нажмите **Add** для добавления формы.

15.Откройте первую форму в режиме дизайнера и перенесите еще одну кнопку на первую форму.

16.В конструкторе дважды щелкните новую кнопку, чтобы открыть окно с кодом обработчика события **Click.** Добавьте в этот метод код, создающий объект класса новой формы и откройте ее с помощью метода `Show()`:

```
nForm myF2 = new nForm();
myF2.Show();
```

17.Постройте и запустите приложение. Кликните новую кнопку и проверьте открытие окна.

Представляет интерес указать конкретное местоположение открытия новой формы, например, требуется так, чтобы вторая форма открывалась рядом – справа стартовой формы.

18.Откройте первую форму в режиме кода и в обработчике открытия новой формы после создания объекта формы укажите свойству `StartPosition` значение `Manual`, а свойству `Location` – вычисляемое значение относительно положения главной формы:

```
nForm myF2 = new nForm();
myF2.StartPosition = FormStartPosition.Manual;
myF2.Location = new Point(this.Location.X + this.Width, this.Location.Y);
myF2.Show();
```

19.Постройте и запустите приложение. Кликните новую кнопку и проверьте открытие окна справа от главной формы.

20.При повторных кликах по кнопке создаются новые экземпляры второй формы. Проверьте это.

Для того, чтобы при повторных кликах не открывались новые экземпляры выполните следующее:

21. В поле класса первой формы объявите ссылку на класс второй формы:

```
nForm myF2;
```

22. В конструкторе первой формы укажите код инициализации этой ссылки:

```
myF2 = new nForm();
```

23. В обработчике события кнопки открытия второй формы удалите код создания ее экземпляра (`nForm myF2 = new nForm()`).

24.Постройте и запустите приложение в режиме отладки. Кликните новую кнопку и проверьте открытие окна справа от главной формы. Закройте новую форму. Повторно кликните кнопку для ее открытия – произойдет генерация исключения **ObjectDisposedException**.

Решить данную проблему можно несколькими путями, например, реализовать обработку исключения **ObjectDisposedException** в классе первой формы или реализовать событие, возникающее перед закрытием второй формы.

25.В обработчике события кнопки открытия второй формы реализуйте перехват и обработку исключения **ObjectDisposedException**:

```
try
{
    myF2.Show();
    myF2.Activate();

}
catch (ObjectDisposedException ex)
{
    myF2 = new nForm();
    myF2.Text = "Повторное создание формы";
    myF2.Show();
    myF2.Activate();
}
```

26. Постройте и запустите приложение. Проверьте, что после закрытия второй формы можно повторно ее открыть, причем заголовок формы будет соответствующим.

Второй вариант решения проблемы будет основан на обработке события, возникающее перед закрытием второй формы – закрытие формы будет подменено его скрытием (Hide).

27.В классе второй формы добавьте обработчик события **nFormClosing** – именно оно возникает непосредственно перед закрытием формы:

```
private void nForm_FormClosing(object sender, FormClosingEventArgs e)
{
        e.Cancel = true;
        Hide();
}
```

28. Постройте и запустите приложение. Проверьте, что теперь после закрытия второй формы – на самом деле она не закрывается, а скрывается (остается в памяти) и можно повторно ее открыть, причем заголовок формы остается прежним.

Теперь при любом закрытии формы (кликом по крестику, командой системного меню или комбинацией Alt+F4) она будет скрываться и оставаться в памяти до закрытия всего приложения.

Для того, чтобы ее можно было бы все-таки закрыть можно реализовать выбор процесса закрытия с помощью элемента «флажок»:

- добавьте его на форму, укажите имя `checkBoxClose`, текст – на ваше усмотрение;
- укажите код считывания состояния в начале метода-обработчика **nForm_FormClosing** и при установке флажка должен быть выход из метода:

```
if (checkBoxClose.Checked ) return;
```

29. Постройте и запустите приложение. Проверьте, что при установке флажка форма действительно закрывается, и повторное открытие формы также возможно, причем заголовок формы будет соответствующим.

Разделить процессы закрытия и скрытия формы можно и другим способом:

- Добавьте на вторую форму новую кнопку – *Закрыть*, добавьте для нее метод-обработчик события клика и в нем вызовите метод закрытия формы `Close();`
- В поле класса формы объявите булевую переменную:

```
private bool nclose = false;
```

- В класс формы добавьте определение метода закрытия формы с модификатором **new**, в этом случае скрывается наследование метода закрытия формы Form:

```
public new void Close()
{
    nclose = true;
    base.Close();
}
```

- В методе-обработчика **nForm_FormClosing** замените проверку установки флажка на проверку булевой переменной:

```
if (nclose) return;
```

30. Постройте и запустите приложение. Проверьте, что клике по кнопке *Закрыть* – форма действительно закрывается.

Упражнение 2. Создание непрямоугольной формы Windows

В этом упражнении вы создадите треугольную форму Windows.

1. Откройте Visual Studio и создайте новый проект Windows Forms. Проект откроется с формой по умолчанию с именем **Form1** в конструкторе.

2. В окне Properties задайте свойству **FormBorderStyle** значение **None,** а свойству **BackColor** значение **Red.** В этом случае форму легче будет увидеть при тестировании приложения.

3. Перетащите кнопку из **Toolbox** в левый верхний угол формы. Задайте свойству **Text** кнопки значение **Close Form.**

4. Дважды щелкните кнопку **Close Form** и добавьте в обработчик события **Button1_Click** вызов метода закрытия формы:

```
Close();
```

5. В конструкторе дважды щелкните форму, чтобы открыть обработчик события **Form1_Load.** Добавьте в этот метод следующий код (он задает области формы треугольную форму указанием многоугольника с тремя углами, более подробно графика будет рассмотрена в работе 6):

```
System.Drawing.Drawing2D.GraphicsPath myPath =
 new System.Drawing.Drawing2D.GraphicsPath();
myPath.AddPolygon(new Point[] { new Point(0, 0),
 new Point(0, this.Height),
new Point(this.Width, 0) });
Region myRegion = new Region(myPath);
this.Region = myRegion;
```

6. Постройте и запустите приложение. Появится треугольная форма.

Упражнение 3. Создание наследуемой формы

Если у вас имеется уже готовая форма, которую вы собираетесь использовать в нескольких приложениях, удобно создать наследуемую (производную) форму. В этом упражнении вы создадите новую форму и унаследуете ее от существующей базовой формы, а затем измените производную форму, настроив ее для конкретной работы.

1. Откройте проект из предыдущего упражнения. Базовой формой для создания производной будет треугольная форма.

2. Для кнопки **Close Form** задайте свойство **Modifiers** как **protected**.

3. Постройте проект.

4. Добавьте производную форму: меню **Project (Проект) | Add Windows Form…(Добавить форму Windows)**, в окне **Categories (Категории)** укажите **Windows Form**, в окне **Templates (Шаблоны)** выберите **Inherited Form (Наследуемая форма).**

5. В окне **Add New Item** в поле **Name** укажите название формы: nForm.cs и нажмите **Add** для добавления формы.

6. В появившемся окне **Inheritance Picker**, в котором отображаются все формы текущего проекта, выберите базовую форму Form1 и нажмите OK.

7. Постройте проект.

8. Откройте форму nForm в режиме конструктора. Проверьте, что она имеет треугольную форму и свойства базовой формы и элемента управления наследованы.

9. Настройте свойства производной формы:
   a. для кнопки:
      i. свойство **Text** – Hello!!!
      ii. свойство **BackColor** – Brown
   b. для формы: свойство **BackColor** – Blue

10. Постройте проект.

11.Задайте производную форму в качестве стартовой, указав в функции `Main` следующий код:

```
Application.Run(new nForm());
```

12.Постройте и запустите приложение. Должна открыться производная форма со своими свойствами. Проверьте, наследуется ли закрытие формы кнопкой.

Упражнение 4. Создание MDI-приложения

В этом упражнении вы создадите MDI-приложение с родительской формой, загружающей и организующей дочерние формы. Также вы познакомитесь с элементом управления **MenuStrip**, который позволяет создать меню формы.

- **Создание нового проекта с базовой формой**

1. Создайте новый проект Windows Forms, укажите имя MdiApplication.
2. Переименуйте файл Form1.cs на ParentForm.cs.
3. Для формы задайте следующие свойства:

Name	ParentForm
Size	420; 320
Text	Parent Form

4. Проверьте, что произошли изменения в функции `Main` так, чтобы форма **ParentForm** стала стартовой.
5. Откройте файл ParentForm.cs в режиме конструктора.
6. Для свойства формы **IsMdiContainer** задайте значение **True.** Таким способом эта форма будет определена как родительская форма MDI.

- **Создание меню для работы с формами**

7. Создайте пункт меню **File:**
   a. Откройте ПИ **Toolbox**, добавьте на форму ЭУ **MenuStrip** и задайте для его свойства **Name** значение **MdiMenu**.
   b. Выделите меню в верхней части формы и задайте имя первого пункта меню **&File**.
   c. Для свойства **Name** пункта меню **File** задайте значение **FileMenuItem.**
   d. Раскройте меню **File**.
   e. Выделите элемент, появившейся под элементом **File,** и задайте его как **&New**.
   f. Для свойства **Name** пункта меню **New** задайте значение **NewMenuItem.**

g. Выделите элемент, появившийся под элементом **New,** и задайте его как **&Exit.**

h. Для свойства **Name** пункта меню **Exit** задайте значение **ExitMenuItem.**

i. Дважды кликните левой кнопкой мыши по пункту меню **Exit** для создания обработчика события **Click**.

j. В обработчик события **Click** для пункта меню **Exit** добавьте следующий код:

```
this.Close();
```

8. Создайте пункт меню **Window:**

a. Переключитесь в режим конструктора.

b. Выделите второй пункт меню справа от **File** и задайте его значением **&Window**.

c. Для свойства **Name** пункта меню **Window** задайте значение **WindowMenuItem.**

d. Раскройте меню **Window**.

e. Выделите элемент, появившейся под элементом **Window,** и задайте для его свойства **Text** значение **&Cascade**.

f. Для свойства **Name** пункта меню **Cascade** задайте значение **WindowCascadeMenuItem.**

g. Выделите элемент, появившийся под элементом **Cascade,** и задайте для его свойства **Text** значение **&Tile.**

h. Для свойства **Name** пункта меню **Tile** задайте значение **WindowTileMenuItem.**

i. Дважды кликните левой кнопкой мыши по пункту меню **Cascade** для создания обработчика события **Click**:

```
this.LayoutMdi (System.Windows.Forms.MdiLayout.Cascade);
```

j. Вернитесь в режим конструктора и дважды кликните левой кнопкой мыши по пункту меню **Tile.**

k. В обработчик события **Click** для пункта меню **Tile** добавьте следующий код:

```
this.LayoutMdi(System.Windows.Forms.MdiLayout.TileHorizontal);
```

9. Реализуйте список открытых окон в меню **Window**:

a. В конструкторе выберите компонент **MdiMenu**. Укажите в свойстве **MdiWindowListItem** имя пункта, созданного для этого – **WindowMenuItem.**

- **Создание дочерней формы**

10. Создайте дочернюю форму:

a. Выберите пункт меню **Project | Add Windows Form.**

b. Задайте имя формы **ChildForm.cs**.

c. Для свойства **Text** формы задайте значение **Child Form**.

d. На ПИ **Toolbox** дважды кликните левой кнопкой мыши по ЭУ **RichTextBox** и задайте для его свойства **Name** значение **ChildTextBox**.

e. Для свойства **Dock** ЭУ **RichTextBox** задайте значение **Fill**.

f. Удалите существующий текст (если он есть) для свойства **Text** ЭУ **RichTextBox** и оставьте его пустым.

g. На ПИ **Toolbox** дважды кликните левой кнопкой мыши по ЭУ **MenuStrip**.

h. Для свойства **Name** ЭУ **MenuStrip** задайте значение **ChildWindowMenu**.

i. Выделите меню в верхней части формы и наберите текст **F&ormat**.

j. Для свойства **Name** пункта меню **Format** задайте значение **FormatMenuItem**, для свойства **MergeAction** установите значение **Insert**, а свойству **MergeIndex** значение **1**. В этом случае меню **Format** будет располагаться после **File** при объединении базового и дочерних меню.

k. Выделите элемент, появившийся под элементом **Format,** и наберите текст **&Toggle Foreground**.

l. Для свойства **Name** пункта меню **Toggle Foreground** задайте значение **ToggleMenuItem**.

m. Дважды кликните левой кнопкой мыши по пункту меню **Toggle Foreground** и добавьте следующий код в обработчик события **Click**:

```
if (ToggleMenuItem.Checked)
{
        ToggleMenuItem.Checked = false;
        ChildTextBox.ForeColor = System.Drawing.Color.Black;
}
else
{
        ToggleMenuItem.Checked = true;
        ChildTextBox.ForeColor = System.Drawing.Color.Blue;
}
```

- **Отображение дочерней формы**

11. Отобразите дочернюю форму в родительской форме:

a. Откройте ParentForm.cs в режиме конструктора.

b. Дважды кликните левой кнопкой мыши по кнопке **New** в меню **File** для создания обработчика события **Click.**

c. Добавьте следующий код для обработчика события **Click** для пункта меню **New:**

```
ChildForm newChild = new ChildForm();
newChild.MdiParent = this;
newChild.Show();
```

- **Работа с приложением**

12. Проверьте работу приложения:
    a. Постройте и запустите приложение.
    b. Когда появится родительская форма, выберите пункт меню **File | New**.
    c. В родительском окне появится новая дочерняя форма. Обратите внимание на то, дочернее меню сливается с родительским и пункты меню упорядочиваются в соответствие со свойством **MergeIndex**, установленным ранее.
    d. Наберите какой-нибудь текст в дочернем окне и воспользуйтесь пунктом меню **Format** для изменения цвета шрифта текста.
    e. Откройте еще несколько дочерних окон.
    f. Выберите пункт меню **Window | Tile**. Обратите внимание на то, что дочерние окна выстраиваются в упорядоченном порядке.
    g. Закройте все дочерние окна.
    h. Обратите внимание на то, что, когда закроется последнее дочернее окно, меню родительской формы изменится, и оттуда исчезнет пункт **Format**.
    i. Для закрытия приложения выберите пункт меню **File | Exit**.
13. Обратите внимание, что заголовок у дочерних окон одинаковый. При создании нескольких документов, например, в Microsoft Word, они называются ДокументN, где N — номер документа. Реализуйте эту возможность:
    a. Откройте код родительской формы и в классе `ParentForm` объявите переменную openDocuments:

    ```
    private int openDocuments = 0;
    ```

    b. К свойству **Text** дочерней формы добавьте счетчик числа открываемых документов (в коде обработчика события **Click** для пункта меню **New)**:

    ```
    newChild.Text = newChild.Text+" "+ ++openDocuments;
    ```

14. Запустите приложение. Теперь заголовки новых документов содержат порядковый номер.

Упражнение 5 (контрольное). Создание пользовательской формы

Для углубления знаний о добавлении и настройке форм Windows выполните следующие задания.

Задание 1. Создайте пользовательскую форму, которая во время выполнения будет иметь овальное очертание. Данная форма должна содержать функциональность, дающую возможность пользователю закрывать ее во время выполнения.

Рекомендация: при разработке формы в виде эллипса используйте следующий код:

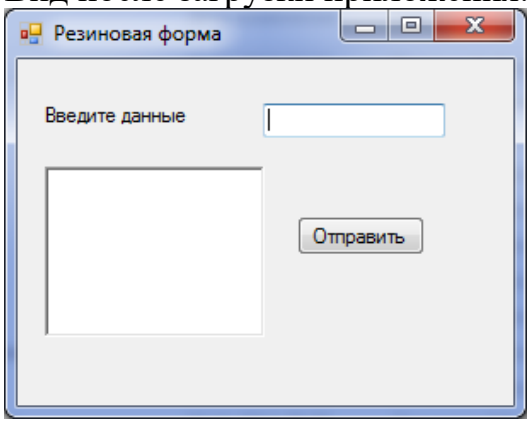
```
// Добавление эллипса, вписанного в прямоугольную форму
// заданной ширины и высоты
myPath.AddEllipse(0, 0, this.Width, this.Height);
```

Задание 2. Создайте приложение с двумя формами и установите вторую форму как стартовую. Сделайте так, чтобы при запуске стартовая форма разворачивалась до максимальных размеров и содержала функциональность, дающую возможность пользователю открыть первую форму, отображающуюся в виде ромба зеленого цвета с кнопкой (в центре ромба) закрытия формы с надписью GREENPEACE.

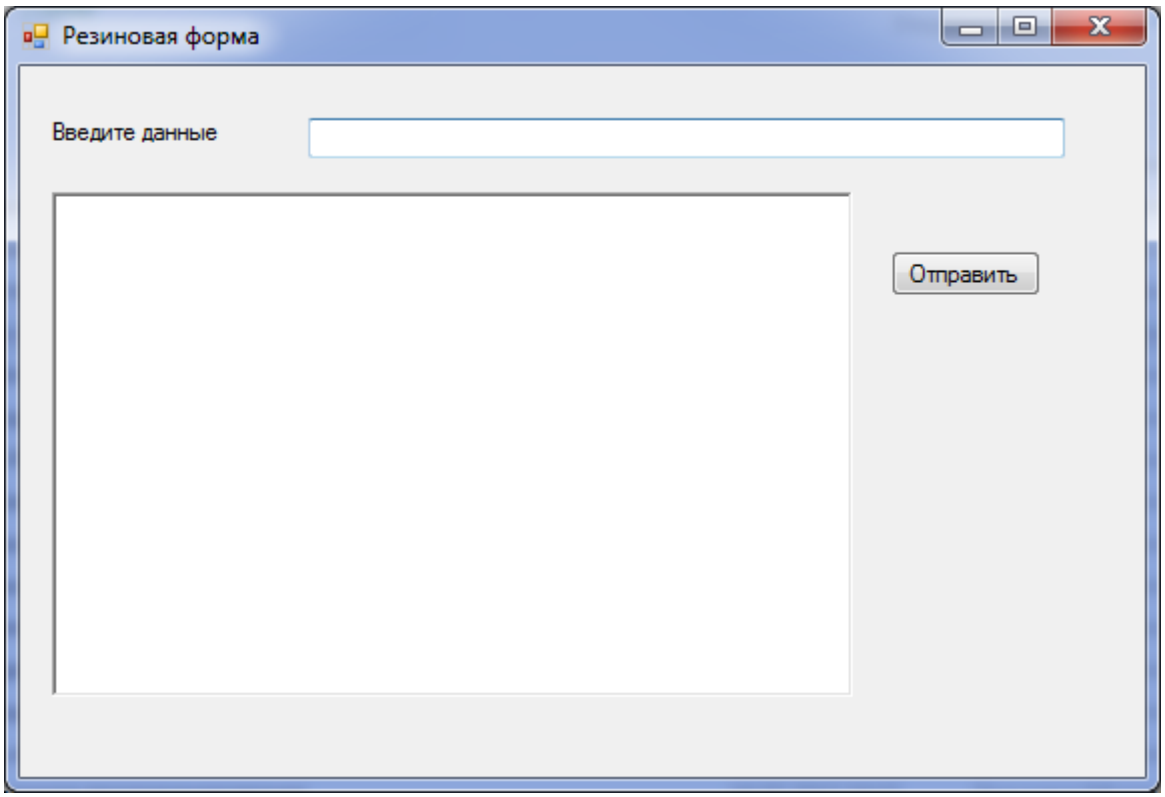
Задание 3. Создайте приложение с “резиновым” интерфейсом: при изменении размера окна программы некоторые элементы растягиваются.

Установите минимальный размер формы в пикселях (на ваше усмотрение), тем самым форму будет невозможно сделать меньше установленного размера.

Вид после загрузки приложения:



Вид после изменения размеров окна:



Указание: для решения задачи воспользуйтесь свойством элементов управления **Anchor**, которое определяет стороны контейнера (в данном случае это форма) (Top, Bottom, Left, Right) к которым привязывается элемент управления.

Лабораторная работа 2. Работа с элементами управления

Элементы управления — это компоненты, объединяющие графический интерфейс с предварительно разработанной функциональностью. Элементы управления представляют собой многократно используемые блоки кода, предназначенные для выполнения определенных задач. Все элементы управления являются производными базового класса **Control**, а значит, тоже используют различные свойства, задающие размер, расположение и другие основные аспекты элементов управления.

Выполнив первое упражнение этого задания, вы создадите простое приложение, отслеживающее события мыши, которые происходят у конкретного элемента управления.

Упражнение 1. Обработка событий Click и MouseMove

1. Создайте новое Windows-приложение. Назовите его WinQuestion.
2. Расположите на форме две кнопки **Button** и надпись **Label** и разместите их по-своему усмотрению.
3. Установите следующие свойства элементов управления и формы:

	Свойство	**Значение**
Form1	FormBorderStyle	Fixed3D

	Свойство	Значение
	Size	350; 200
	Text	Насущный вопрос
label1	Text	Вы довольны своей зарплатой?
Button1	Name	btnyes
	Text	Да
Button2	Name	btnno
	Text	Нет

4. Щелкните дважды по кнопке "Да". В обработчике этой кнопки добавьте следующий код:

```
MessageBox.Show("Мы и не сомневались, что Вы так думаете!");
```

5. Выделите кнопку "Нет". В окне Properties переключитесь в окно событий и дважды щелкните в поле MouseMove.
6. В обработчике этого события добавьте код для связывания движения мыши с координатами кнопки и указания координат, куда кнопка будет перемещаться:

```
btnno.Top -= e.Y;
btnno.Left += e.X;
if (btnno.Top < -10 || btnno.Top > 100)
btnno.Top = 60;
if (btnno.Left < -80 || btnno.Left > 250)
btnno.Left = 120;
```

7. Запустите приложение и нажмите на каждую из кнопок.
8. Проверьте, что с помощью кнопки табуляции можно «поймать» кнопку. Запретите это с помощью установки для кнопки `btnno` свойству **TabStop** значение False.

Упражнение 2. Работа со списками

Основными элементами управления списком являются **ListBox, ComboBox** и **CheckedListBox**. Несмотря на некоторые отличия во внешнем виде и разные функциональные возможности, они одинаково формируют и представляют списки данных и включают в себя коллекцию **Items,** которая систематизирует элементы, содержащие один из этих элементов управления.

ListBox – самый простой элемент управления списка. Он служит главным образом для отображения простого списка элементов в пользовательском интерфейсе, по которому легко перемещаться.

CheckedListBox – помечаемый список. Является разновидностью простого списка. Его дополнительное достоинство — в наличии чекбоксов рядом с каждым элементом списка. Пользователь имеет возможность отметить один или несколько элементов списка, выставив напротив его флажок.

ComboBox – выпадающий список. Постоянно на форме представлено только одно значение этого списка. При необходимости пользователь может

раскрыть список и выбрать другое интересующее его значение или ввести собственное.

- **Создание приложения, использующее список**

1. Создайте новый проект Windows Forms, укажите имя TestList.
2. Добавьте на форму следующие элементы управления:
    a. **GroupBox**,
    b. **CheckedListBox** (поместите в **GroupBox**)
    c. **ComboBox**
    d. три элемента **Button**.
3. Установите следующие свойства формы и элементов управления:

Элемент управления	**Свойство**	**Значение**
Form1	FormBorderStyle	Fixed3D
	Text	Работа со списками
	Size	400;300
groupBox1	Text	Список участников
CheckedListBox	Name	memberList
ComboBox	Name	peopleList
	Text	
Button1	Name	buttonAdd
	Text	Добавить
Button2	Name	buttonDelete
	Text	Удалить
Button3	Name	buttonSort
	Text	Сортировать

4. Возможный вид приложения изображен на рис. 2.1:

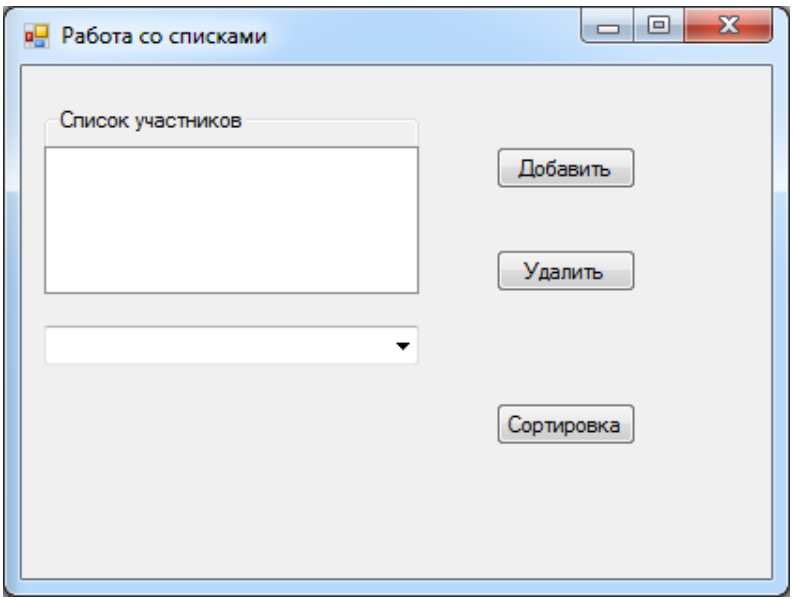


Рис. 2.1 Готовое приложение

5. Проинициализируйте элемент управления **ComboBox** списком предполагаемых участников. Для этого в окне свойств peopleList выберите свойство **Items**. Откройте окно **String Collection Editor,** нажав на кнопку с тремя точками в поле **Items**. Добавьте в окно Ф.И.О. нескольких участников. Нажмите OK.
6. Добавьте обработчики для кнопок **Добавить** и **Удалить,** два раза щелкнув левой кнопкой мыши по каждой из кнопок.
7. В тело обработчика события кнопки **Добавить** вставьте следующий код:

```
if (peopleList.Text.Length != 0)
 {
 memberList.Items.Add(peopleList.Text);
 }
       else MessageBox.Show("Выберите элемент из списка или введите
новый");
```

8. Для реализации удаления элементов из списка введите код в тело обработчика события кнопки **Удалить**:

```
while (memberList.CheckedIndices.Count > 0)
       memberList.Items.RemoveAt(memberList.CheckedIndices[0]);
```

9. Для реализации сортировки элементов введите код в тело обработчика события кнопки **Сортировать:**

```
memberList.Sorted = true;
```

10. Откомпилируйте и запустите приложение. Заполните список участников, выбирая их из элемента **ComboBox.** Запишите новые данные в этот элемент и добавьте их в список. Отсортируйте список участников.

Использование XML-файла в качестве источника данных

1. Добавьте в проект XML-файл: в контекстном меню проекта выберите команду **Добавить**, а затем **Создать элемент**.

2. В окне добавления нового элемента выберите XML-файл, укажите его имя, например, XMLData.
3. В открывшемся файле добавьте следующие данные:

```
<Head>
 <People>Иванов Сергей Петрович</People>
  <People>Петров Иван Сергеевич</People>
  <People>Сидоров Петр Никитич</People>
  <People>Дано Игорь Петрович</People>
  <People>Моргунов Анатолий Таросович</People>
</Head>
```

4. Сохраните изменения в созданном файле.
5. Перейдите в режим дизайнера формы и добавьте справа от выпадающего списка кнопку с надписью **Загрузить данные**.
6. Добавьте обработчик клика по новой кнопке и в нем реализуйте загрузку данных из XML-файла:
   a. добавьте требуемое пространство имен:

```
using System.Xml;
```

   b. если требуется очистить компонент перед загрузкой, то вызовите метод **Clear**:

```
peopleList.Items.Clear();
```

   c. создайте объект класса **FileStream**, представляющий содержимое xml-файла:

```
FileStream fStream =
   new FileStream("..\\..\\XMLData.xml", FileMode.Open,
       FileAccess.Read, FileShare.ReadWrite);
```

   d. создайте объект класса **XmlDocument** и заполните его данными

```
XmlDocument xmlDoc = new XmlDocument();
xmlDoc.Load(fStream);
```

   e. с помощью цикла for заполните компонент значениями внутреннего текста xml-документа:

```
for (int i = 0; i < xmlDoc.DocumentElement.ChildNodes.Count; i++)
{
peopleList.Items.Add(xmlDoc.DocumentElement.ChildNodes[i].InnerText);
}
```

   f. закройте файловый поток:

```
fStream.Close();
```

7. Откомпилируйте и запустите приложение. Протестируйте его работу.

Упражнение 3. Создание и использование элемента управления ToolStrip

ToolStrip — это элемент управления в .NET Framework, разработанный с целью упрощения создания пользовательских панелей инструментов, которые выглядят и работают, как панели инструментов Microsoft Office и Microsoft Internet Explorer. Используя элемент управления **ToolStrip**, вы можете быстро разрабатывать легконастраиваемые панели инструментов профессионального вида.

- **Добавление на форму шаблона панели инструментов**

1. Откройте проект MdiApplication.
2. Откройте форму **ParentForm** в режиме конструктора.
3. Добавьте на форму ЭУ **ToolStrip**.
4. На форме откройте выпадающий список ЭУ **ToolStrip** и выберите **button** – добавится элемент ***toolStripButton1***. В панели инструментов он представлен в виде кнопки с рисунком, обозначающим функцию, которую этот элемент содержит.
5. Снова откройте выпадающий список ЭУ **ToolStrip** и выберите **Separator** – добавится элемент, который отделяет одни элементы панели инструментов от других.
6. Справа от разделителя добавьте еще две кнопки – элементы ***toolStripButton2*** и ***toolStripButton3.***
7. В итоге вы должны получить три кнопки, отделенные одним разделителем.

- **Отображение рисунка на элементах панели управления**

8. Выберите первую кнопку. Убедитесь, что в окне Properties свойству **DisplayStyle** задано значение **Image.**
9. В окне Properties выберите изображение элемента управления, щелкнув свойство **Image** и выбрав соответствующее изображение или путь к нему в диалоговом окне **Select Resource.** Если у Вас есть готовые файлы подходящих изображений, то выберите их, в противном случае укажите любой рисунок из папки *Мои рисунки*.
10. Повторите предыдущие два пункта для остальных кнопок.

- **Создание графических изображений кнопок панели инструментов**

11. Если Вас не устраивает вид готовых рисунков, то можно для придания кнопкам графических изображений самостоятельно их создать. Для работы с изображениями в среде разработки существует **Image Editor**. Он позволяет создать изображения с использованием простейших инструментов.

12. Для создания файла с изображением выберите меню **File | New | File...** В появившемся окне **New File** выберите тип файла “Файл точечного рисунка”, нажмите **Open**. Появится пустое изображение с дополнительной панелью управления. В основном меню появится новый пункт меню **Image.** Для отображения панели с палитрой компонент выберите в меню пункт **Image | Show Colors Window (Показать окно выбора цвета).**
13. Создайте по своему усмотрению изображение для кнопки **New**, например, в виде белого листа и сохраните изображение в файл с именем Icon_New.bmp в каталог с решением.
14. Повторите действия для создания иконок для других кнопок, например, в виде нарисованных распылителем букв ***C*** и ***T***. Сохраните изображения в каталог с решением в файлы с именами Icon_Windows_Cascade.bmp и Icon_Windows_Title.bmp соответственно.
15. Выполните действия для указания новых изображений этим кнопкам.
16. Сохраните и запустите проект. Проверьте вид и работоспособность кнопок.

➢ **Добавление обработчиков событий для кнопок**

17. Добавьте обработчик события **Click** объекта ***toolStrip1***, щелкнув два раза указателем мыши по имени события **Click** на закладке событий в окне свойств. В программу добавится функция `toolStrip1_ItemClicked` как обработчик события, происходящего при нажатии кнопки на панели инструментов.
18. В окне Properties для ***toolStripButton1*** в свойстве **Tag** запишите **NewDoc.** Аналогично укажите для ***toolStripButton2*** и ***toolStripButton3*** для свойства **Tag** значения **Cascade** и **Title** соответственно.
19. Укажите для кнопок всплывающие подсказки в свойстве **ToolTipText**: Create new document, Windows cascade и Windows title.
20. В обработчике события **Click** объекта `toolStrip1_ItemClicked` добавьте код, который будет реализовывать различные действия в зависимости от нажимаемой кнопки:

```
switch(e. ClickedItem.Tag.ToString())
        {
        case "NewDoc":
              ChildForm newChild = new ChildForm();
              newChild.MdiParent = this;
              newChild.Show();
              newChild.Text = newChild.Text+" "+ ++openDocuments;
              break;
        case "Cascade":
this.LayoutMdi (System.Windows.Forms.MdiLayout.Cascade);
              break;
        case "Title":
this.LayoutMdi (System.Windows.Forms.MdiLayout.TileHorizontal);
        break;
```

```
}
```

21. Откомпилируйте и запустите приложение. Проверьте работоспособность кнопок.

Упражнение 4. Использование элемента управления StatusStrip

Элемент управления **StatusStrip** применяется в программах для вывода информации в строку состояния — небольшую полоску, расположенную внизу приложения. В этом упражнении вы добавите к приложению MdiApplication строку состояния, на которой показывается вариант ориентации окон и выводится текущая дата.

1. Откройте проект MdiApplication.
2. Увеличьте размер формы **ParentForm** до значения (450;350).
3. Добавьте на форму **ParentForm** элемент управления **StatusStrip**.
4. Удалите содержимое поля свойства **Text**.
5. Щелкните на кнопку выпадающего списка панели и выберите **StatusLabel.** Добавится элемент ***toolStripStatusLabel1*** – первая панель для отображения.
6. Создайте еще одну панель аналогичным способом – ***toolStripStatusLabel2*** и установите им следующие свойства:

Первая панель	**Text**	Status
	Name	spWin
Вторая панель	**Text**	Data
	Name	spData

7. Для отображения информации на первой панели вставьте в соответствующие обработчики команд меню и кнопок на панели инструментов следующую строку кода:
   a. Для каскадной ориентации:

```
spWin.Text="Windows is cascade";
```

   b. Для горизонтальной ориентации:

```
spWin.Text="Windows is horizontal";
```

8. Для отображения даты на второй панели в конструкторе формы **ParentForm** добавьте код:

```
public ParentForm()
{
InitializeComponent();
// Свойству Text панели spData устанавливается текущая дата
spData.Text = Convert.ToString(System.DateTime.Today.ToLongDateString());
}
```

9. Откомпилируйте и запустите приложение. Проверьте работоспособность панели состояния.

Упражнение 5. Работа с контейнерными элементами управления

Контейнерные элементы управления — это специализированные элементы управления, выступающие в роли настраиваемого вместилища для других элементов управления. К контейнерным элементам управления относятся **Panel** и **GroupBox**. Они предоставляют форме логические и физические подразделы, которые могут группировать другие элементы управления в единообразные подгруппы пользовательского интерфейса. Например, элемент управления **GroupBox** содержит в себе набор связанных элементов управления **RadioButton**. Контейнерные элементы управления помогут вам создать ощущение стиля или информационного потока в пользовательском интерфейсе и позволят согласованно управлять элементами управления, которые содержатся в них.

Выполнив это упражнение, вы научитесь создавать формы Windows с использованием различных контейнерных элементов управления.

- **Создание проекта с возможностью группировки элементов на вкладках**

1. Откройте Visual Studio и создайте новый проект Windows Forms. Назовите его WinContainer.

2. Перетащите из Toolbox в форму элемент управления **TabControl.** В окне Properties задайте свойству **Dock** значение **Fill.**

3. В окне Properties выберите свойство **TabPages,** чтобы открыть TabPage Collection Editor. Добавьте вкладки так, чтобы их стало всего пять. Задайте свойствам **Text** этих пяти элементов управления **TabPage** значения **GroupBox**, **Panel, FlowLayoutPanel**, **TableLayoutPanel** и **SplitContainer**. Щелкните ОК.

- **Настройка контейнерного элемента GroupBox**

4. В форме выберите вкладку GroupBox. Перетащите элемент управления **GroupBox** из Toolbox в элемент управления **TabPage.**

5. Перетащите в **GroupBox** два элемента управления **RadioButton.**

6. Добавьте на вкладку GroupBox вне элемента управления **GroupBox** кнопку (элемент управления **Button**). Для кнопки свойство **Text** сделайте пустым, а свойству **Name** укажите значение `but`.

7. Дважды кликните по кнопке и добавьте код обработчика события установки надписи на кнопке в зависимости от выбранного переключателя (**RadioButton**):

```
if (radioButton1.Checked == true)
       this.but.Text = "First";
             else if (radioButton2.Checked == true)
                   this.but.Text = "Second";
```

- **Настройка элемента Panel**

8. Выберите в форме вкладку Panel. Перетащите элемент управления **Panel** из Toolbox в элемент управления **TabPage.** Для элемента **Panel** задайте свойству **Dock** значение **Fill.**

9. Перетащите четыре элемента управления **Button** из Toolbox в элемент управления **Panel.**

10.Свойству **AutoScroll** установите значение **True**, в этом случае элемент управления **Panel** будет отображать полосы прокрутки, если элементы находятся за пределами видимых границ.

- **Настройка элемента FlowLayoutPanel**

11.Выберите в форме вкладку FlowLayoutPanel. Перетащите элемент управления **FlowLayoutPanel** из Toolbox в элемент управления **TabPage**. Задайте значение **Fill** свойству **Dock** элемента управления **FlowLayoutPanel**.

12.Перетащите четыре элемента управления **Button** из Toolbox в элемент управления **Panel.** Обратите внимание на размещение добавляемых элементов: по умолчанию порядок следования элементов управления в **FlowLayoutPanel** — слева направо. Это значит, что элементы управления, расположенные в **FlowLayoutPanel***,* будут находиться в левом верхнем углу и размещаться вправо до тех пор, пока не достигнут края панели. Такое поведение контролируется свойством *FlowDirection,* которому может быть задано четыре значения заливки в **FlowLayoutPanel***: LeftToRight* — по умолчанию, *RightToLeft* — справа налево, *TopDown* — сверху вниз и *BottomUp* — снизу-вверх.

13.Дважды щелкните кнопку **button5** и добавьте в обработчик события `button5_Click` следующий код:

```
flowLayoutPanel1.SetFlowBreak(button6, true);
```

- **Настройка элемента TableLayoutPanel**

14.Выберите конструктор формы (если это необходимо). В форме выберите вкладку TableLayoutPanel. Перетащите элемент управления **TableLayoutPanel** из Toolbox в **TabPage**. Задайте свойству **CellBorderStyle** (определяет вид ячеек таблицы и их поведение) значение **Inset***,* а свойству **AutoScroll** — **True**.

15.Перетащите элемент управления **Button** из Toolbox в левую верхнюю ячейку элемента управления **TableLayoutPanel**.

16. Дважды щелкните **Button9** и добавьте в обработчик события `Button9_Click` следующий код:

```
Button aButton = new Button();
tableLayoutPanel1.Controls.Add(aButton, 1, 1);
```

- **Настройка элемента SplitContainer**

17.В конструкторе выберите вкладку SplitContainer. Перетащите элемент управления **SplitContainer** из Toolbox в **TabPage**. Задайте свойству **BorderStyle** значение **Fixed3D**.

18.Перетащите два элемента управления **Button** из Toolbox в Panel1 элемента управления **SplitContainer**. Задайте свойствам **Text** этих кнопок

значения *Fix/Unfix Panel1* и *Fix/Unfix Splitter.* Измените размеры кнопок так, чтобы отображался текст.

19.Добавьте кнопку в Panel2 и задайте свойству **Text** значение *Collapse/Uncollapse Panel1.* Измените размеры кнопок так, чтобы отображался текст.

20.Дважды щелкните кнопку *Fix/Unfix Panel1* и добавьте в обработчик события *Click* следующий код:

```
if (splitContainer1.FixedPanel == FixedPanel.Panel1)
        splitContainer1.FixedPanel = FixedPanel.None;
      else
        splitContainer1.FixedPanel = FixedPanel.Panel1;
```

21.Дважды щелкните кнопку *Fix/Unfix Splitter* и добавьте в обработчик события *Click* следующий код

```
splitContainer1.IsSplitterFixed = !(splitContainer1.IsSplitterFixed);
```

22.Дважды щелкните кнопку *Collapse/Uncollapse Panell* и добавьте в обработчик
события *Click* следующий код:

```
splitContainer1.Panel1Collapsed = !(splitContainer1.Panel1Collapsed);
```

23.Постройте и запустите приложение.

24.На вкладке **GroupBox** поочередно выбирайте переключатели следите за изменением надписи на кнопке.

25.На вкладке **Panel** измените размер формы с помощью мыши. Проверьте, появились ли полосы прокрутки.

26.На вкладке **FlowLayoutPanel** измените размер формы с помощью мыши. Просмотрите, что автоматически изменилась компоновка. Щелкните кнопку **button5** и проверьте, прервалась ли последовательность на элементе управления **button6** (это было реализовано вызовом метода *SetFlowBreak*).

27.На вкладке **TableLayoutPanel** щелкните кнопку **button9**, добавится новая кнопка.

28. На вкладке **SplitContainer** измените размеры формы, а также размеры каждой панели, передвинув *Splitter.* По очереди щелкайте каждую кнопку и смотрите, как это отражается на возможности элемента управления изменять свои размеры.

Упражнение 6. Реализация простого интерфейса ввода-вывода

Выполнив это упражнение, вы научитесь использовать элементы управления формы для ввода и вывода информации об объектах.

- **Создание проекта**

1. Создайте новый проект Windows Forms. Назовите его BiblWorm.

2. Добавьте в проект папку **Bibl**, содержащую классы, моделирующие библиотеку: Items, IPubs, Item и Magazine. Изучите их содержание, обратите

внимание на абстрактный класс Items и применение интерфейсов IComparable и IPubs.

3. Изучите внешний вид готовой формы на рис. 6.1. Указанные в следующем пункте элементы необходимо будет разместить в соответствии с рисунком.

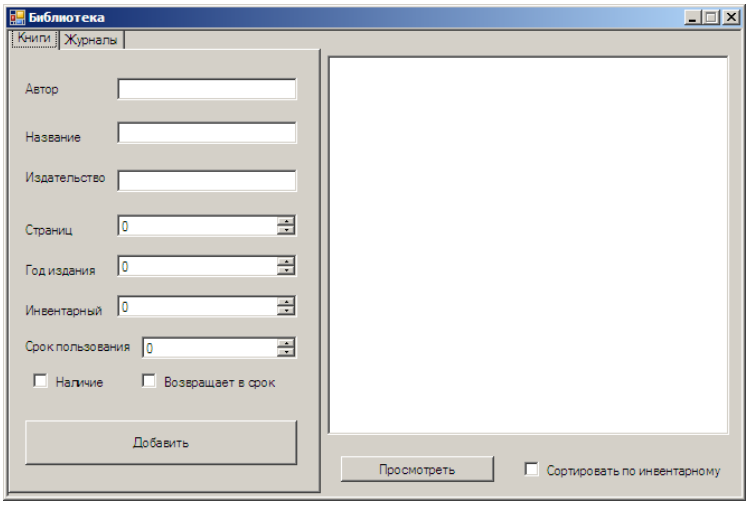


Рис. 6.1 Готовая форма приложения

➢ **Реализация интерфейса приложения**

4. Для формы установите размер – 670;450 и заголовок – Библиотека. Настройте форму так, чтобы нельзя было изменять границы окна и разворачивать его.

5. Для реализации раздельного ввода данных о книгах и журналах добавьте на форму элемент **TabControl** и с помощью свойства **TabPages** добавьте две вкладки**:** первую назовите – Книги, вторую – Журналы.

6. Для ввода текстовой информации об авторе, названии книги, издательстве используйте элемент **TextBox**, а для числовых данных (количества страниц, год издания, инвентарный номер и срок пользования) – элемент **NumericUpDown** (для этого элемента укажите в окне свойств допустимый диапазон значений – свойства Maximum и Minimum для каждого элемента по своему усмотрению).

7. Для реализации логических операций (наличие книги, возвращение в срок и сортировка) добавьте три флажка **CheckBox**.

8. Для вывода перечня книг (будет реализован в виде списка) разместите в правой части формы элемент **RichTextBox**.

9. Добавьте две кнопки – Добавить и Просмотреть.

➢ **Реализация связи элементов формы и данных**

10.В поле класса формы Form1 (Form1.cs) создайте свойства, соответствующие элементам управления формы**.** Таким образом, в дальнейшем связь данных будет реализована с помощью этих свойств.

```
public string Author // автор
{
```

```
        get { return textBox1.Text; }
        set { textBox1.Text = value; }
    }

    public string Title // Название
    {
        get { return textBox2.Text; }
        set { textBox2.Text = value; }
    }

    public string PublishHouse // Издательство
    {
        get { return textBox3.Text; }
        set { textBox3.Text = value; }
    }

    public int Page // Количество страниц
    {
        get { return (int)numericUpDown1.Value; }
        set { numericUpDown1.Value = value; }
    }

    public int Year // Год издания
    {
        get { return (int)numericUpDown2.Value; }
        set { numericUpDown2.Value = value; }
    }

    public int InvNumber // Инвентарный номер
    {
        get { return (int)numericUpDown3.Value; }
        set { numericUpDown3.Value = value; }
    }

    public bool Existence // Наличие
    {
        get { return checkBox1.Checked; }
        set { checkBox1.Checked = value; }
    }

    public bool SortInvNumber // Сортировка по инвентарному номеру
    {
        get { return checkBox2.Checked; }
        set { checkBox2.Checked = value; }
    }

    public bool ReturnTime // Возвращение в срок
    {
        get { return checkBox3.Checked; }
        set { checkBox3.Checked = value; }
    }

    public int PeriodUse // Инвентарный номер
    {
        get { return (int)numericUpDown4.Value; }
        set { numericUpDown4.Value = value; }
    }
```

11.В поле класса формы Form1 (Form1.cs) создайте список для хранения единиц хранения библиотеки:

```
List<Item> its = new List<Item>();
```

12.Добавьте обработчик события клика по кнопке **Добавить**.

Сначала создайте объект класса **Book**, передав в конструктор свойства формы:

```
private void button1_Click(object sender, EventArgs e)
{
    Book b = new Book(Author, Title, PublishHouse,
        Page, Year, InvNumber, Existence);
```

Далее в этом обработчике реализуйте проверку возврата книги в срок:

```
    if (ReturnTime)
        b.ReturnSrok();
```

И расчет стоимости с учетом срока пользования книгой:

```
    b.PriceBook(PeriodUse);
```

Теперь добавьте книгу в список:

```
    its.Add(b);
```

В конце обработчика очистите поля ввода для новой информации (будьте внимательны, укажите значения, входящие в допустимый диапазон, который вы настраивали ранее):

```
    Author = Title = PublishHouse = "";
    Page = InvNumber = PeriodUse = 0;
    Year = 2020;
    Existence = ReturnTime = false;
```

13.Добавьте обработчик события клика по кнопке **Просмотреть**:

В обработчике сначала проверьте состояние флажка сортировки и в случае его установки отсортируйте список:

```
    if (SortInvNumber)
        its.Sort();
```

Для вывода информации создайте строку класса **StringBuilder** и с помощью цикла постройте строку и информацией о единице хранения:

```
    StringBuilder sb = new StringBuilder();
    foreach (Item item in its)
    {
        sb.Append("\n" + item.ToString());
    }
```

После построения строки выведете ее в элемент richTextBox1:

```
    richTextBox1.Text = sb.ToString();
```

- **Тестирование приложения**

14.Постройте и запустите приложение.

15. Заполните поля ввода некоторой соответствующей информацией и нажмите кнопку **Добавить**.

16. Повторите ввод новой информацией несколько раз.

17.Установите флажок сортировки и щелкните кнопку **Просмотреть** для вывода списка книг в правое поле. Вывод данных должен произойти в отсортированном виде по инвентарному номеру.

Упражнение 7. Добавление и удаление элементов управления в режиме работы приложения

При размещении на форме элемента управления в режиме дизайна, среда создает код, описывающий этот элемент. Если назначить в обработчике заданного элемента управления генерацию аналогичного кода, то в

запущенном приложении можно будет добавлять на форму или удалять элементы, активизируя этот обработчик.

Для работы с элементами управления используется объект **ControlsCollection**, содержащий ряд методов, основные из которых будут использованы в данном упражнении.

1. Создайте новое приложение и назовите его RegistrationForm.
2. Добавьте на форму три надписи, два текстовых поля, кнопку, элементы **CheckBox** и **GroupBox**
3. Установите следующие значения свойств формы:

	Свойство	**Значение**
Form1	FormBorderStyle	Fixed3D
	Text	Регистрация
	Size	400;310

4. Элементы управления разместите и настройте согласно рис. 7.1:

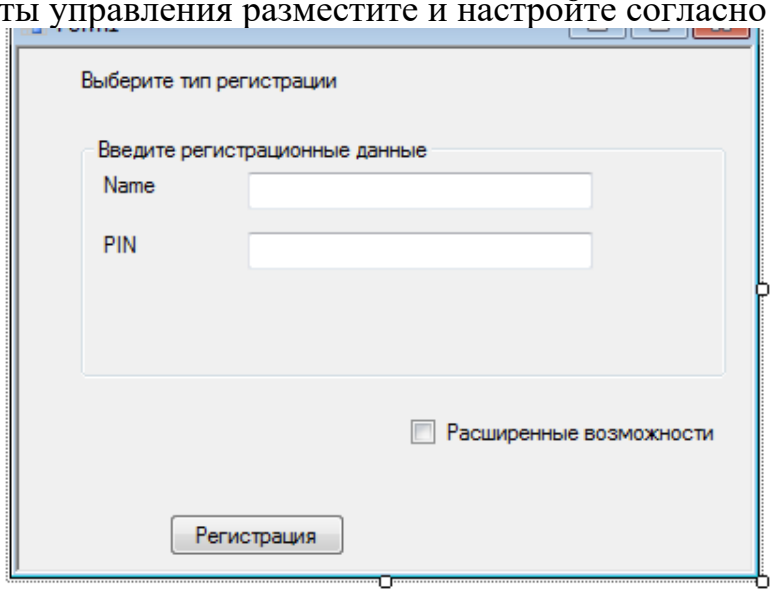


Рис. 7.1 Готовое приложение

5. Для реализации возможности добавления и удаления элементов в процессе выполнения программы реализуйте обработчик события CheckedChanged: щелкните дважды на элементе `checkBox1` и добавьте следующий код:

```
if (checkBox1.Checked == true)
 {
       Label lbl = new Label();
       lbl.Location = new System.Drawing.Point(16, 96);
       lbl.Size = new System.Drawing.Size(32, 23);
       lbl.Name = "labelll";
       lbl.TabIndex = 2;
       lbl.Text = "PIN2";
       groupBox1.Controls.Add(lbl);
       TextBox txt = new TextBox();
       txt.Location = new System.Drawing.Point(96, 96);
       txt.Size = new System.Drawing.Size(184, 20);
       txt.Name = "textboxx";
```

```
            txt.TabIndex = 1;
            txt.Text = "";
            groupBox1.Controls.Add(txt);
      }
      else{
            }
```

6. Откомпилируйте и запустите приложение. Проверьте, что при установке флажка в ЭУ `checkBox` “Расширенные возможности” на форме появляется надпись и поле ввода для дополнительных данных.

7. Для удаления ЭУ с формы могут применяться методы: **Clear** (удаление всех элементов из коллекции), **Remove** (удаление элемента из коллекции) и **RemoveAt** (удаление элемента по заданному индексу). В тело оператора `else` добавьте код для удаления ЭУ по индексу:

```
     int lcv;
lcv = groupBox1.Controls.Count;// определяется количество элементов
            while (lcv > 4)
            {
            groupBox1.Controls.RemoveAt(lcv - 1);
            lcv -= 1;
            }
```

8. Запустите приложение. Убедитесь, что при включении “Расширенные возможности” дополнительные элементы появляются на форме, а при выключении – исчезают.

Упражнение 8. Проверка вводимых значений. События KeyPress и Validating. Элемент управления ErrorProvider

При внесении значений параметров пользователем во многих случаях требуется проверять вводимый текст по заданным критериям. Например, регистрационный номер, телефон не должны содержать букв, поле имени - цифр. В этом упражнении рассматриваются реализации проверок, которые можно осуществлять, используя встроенные события текстового поля.

- **Использование события KeyPress**

1. Откройте приложение RegistrationForm.

2. Выделите поочередно текстовые поля **TextBox1** и **TextBox2,** в окне **Properties** создайте обработчики события **KeyPress**, возникающего при нажатии любой клавиши в поле.

3. В тело обработчика события **KeyPress** для текстового поля **TextBox1** укажите следующий код (для элемента **TextBox1** недопустимыми значениями будут цифры):

```
     if (char.IsDigit(e.KeyChar) )
     {
     e.Handled = true;
      MessageBox.Show("Поле Name не может содержать цифры");
     }
```

4. Для элемента **TextBox2**, наоборот, недопустимыми значениями будут буквы, в обработчике события **KeyPress** для текстового поля **TextBox2** укажите код:

```
     if (!char.IsDigit(e.KeyChar) )
```

```
{
e.Handled = true;
MessageBox.Show("Поле PIN не может содержать буквы");
 }
```

5. Откомпилируйте и запустите приложение. Попробуйте ввести в поле **Name** цифры, в поле **PIN** – буквы.

6. Для защиты текстового поля, появляющегося при установке галочки в чекбоксе "Расширенные возможности", необходимо вручную определить событие **KeyPress.** В обработчике события CheckedChanged для элемента **CheckBox1** укажите код:

```
txt.KeyPress+= new
System.Windows.Forms.KeyPressEventHandler(this.textBox2_KeyPress);
```

7. Запустите и протестируйте приложение.

- **Применение события Validating**

Событие **KeyPress** блокирует часть клавиатуры. Другим способом проверки является событие **Validating**, позволяющее работать с клавиатурой, но блокирующее другие действия пользователя.

8. Закомментируйте обработчик элемента **TextBox2.**

9. В режиме дизайна формы в окне Properties элемента **TextBox2** создайте обработчик события **Validating** и запишите следующий код:

```
if(textBox2.Text ==="")
 {
       e.Cancel=false;
 }
 else
 {
       try
       {
             double.Parse(textBox2.Text);
             e.Cancel = false;
       }
       catch
       {
             e.Cancel = true;
             MessageBox.Show("Поле PIN не может содержать буквы");
       }
 }
```

10. Запустите приложение. При переключении фокуса ввода или нажатии на кнопку "регистрация" происходит событие **Validating**.

- **Применение элемента управления ErrorProvider**

Элемент управления **ErrorProvider** удобно применять, когда нужно выводить небольшую иконку в случае ошибки ввода.

11. В режиме дизайна из окна **ToolBox** перенесите на форму элемент управления **ErrorProvider**.

12. В коде формы в обработчике `textBox1_KeyPress` добавьте следующую строку:

```
errorProvider1.SetError(textBox1, "Must be letter");
```

13. Запустите приложение. При ошибке ввода появляется мигающая иконка уведомления, при наведении на нее всплывает поясняющее сообщение об ошибке.

Упражнение 9 (контрольное). Реализация интерфейса

Добавьте в проект упражнения 6 реализацию вкладки для ввода журналов.

Лабораторная работа 3. Создание элементов управления

В дополнение к уже существующим элементам управления можно разрабатывать собственные, чтобы обеспечить для своих приложений специализированную функциональность.

Существует три вида разрабатываемых пользователем элементов управления:

- **составные (composite)**, которые создаются при объединении других элементов управления Windows Forms;
- **специализированные (custom)**, создаваемые с “нуля” и предоставляющие собственный код для прорисовки;
- **расширенные (extended)**, которые добавляют функциональность к уже существующему элементу управления Windows Forms.

Упражнение 1. Создание составного элемента управления

Составные элементы управления наследуются от класса **UserControl,** Он предоставляет базовый уровень функциональности, обеспечивающий добавление других элементов управления, а также свойств, методов и событий.

Особенность при разработке элементов – класс **UserControl** имеет собственный конструктор, позволяющий использовать в Visual Studio перетаскивание дополнительных элементов управления из **Toolbox** на поверхность конструктора и настраивать их.

В этом упражнении вы создадите составной элемент управления, действующий как цифровые часы. В него вы добавите элемент управления **Label,** отображающий правильное время, и компонент **Timer,** каждую секунду обновляющий **Label.** Предоставив свойство **Enabled** элемента управления **Timer,** вы дадите пользователям возможность включать и отключать часы.

➢ **Разработка составного элемента управления**

1. Создайте в Visual Studio новое приложение Windows Forms. Назовите его **WinTimer1**.

2. В меню **Project** выберите **Add User Control (Добавить пользовательский элемент управления)** и щелкните **Add** в диалоговом окне **Add New Item (Пользовательский элемент управления)**. Укажите имя **UserControlTimer** и нажмите **Добавить**. К вашему проекту будет добавлен

пустой пользовательский элемент управления, который откроется в конструкторе.

3. Из **Toolbox** перетащите **Label** в пользовательский элемент управления.

4. Для ЭУ **Label** удалите текст из свойства **Text.**

5. Из **Toolbox (**группа **Компоненты)** перетащите в пользовательский элемент управления компонент **Timer.**

6. Измените размеры пользовательского элемента управления так, чтобы он был приблизительно равен размеру элемента управления **Label.**

7. В окне **Properties** компонента **Timer** присвойте свойству **Interval** значение **1000** и свойству **Enabled** значение **True.**

8. Добавьте обработчик события **Tick** – для этого дважды щелкните компонент **Timer,** чтобы открыть в окне кода обработчик события **Timer.Tick** и добавьте следующий код обработчика события:

```
label1.Text = DateTime.Now.ToLongTimeString();
```

9. Для класса **UserControlTimer** в окне кода составного элемента добавьте следующее объявление свойства:

```
public bool TimeEnabled
{
    get { return timer1.Enabled; }
    set { timer1.Enabled = value; }
}
```

10. В меню **File** выберите **Save All**, чтобы сохранить ваше решение.

11. В меню **Build** выберите **Build Solution**.

12. Откройте форму **Forml** в режиме конструктора и проверьте, что на панели **Toolbox** появился новый компонент **UserControlTimer.**

➢ **Применение составного элемента управления**

13. Перенесите из панели **Toolbox** компонент **UserControlTimer** на поверхность формы **Forml.**

14. Проверьте, что на форме появился экземпляр вашего пользовательского элемента управления и начал отсчитывать каждую секунду – при необходимости измените его размер для лучшей наглядности.

15. Обратите внимание, что вы можете остановить работу компонента, присвоив свойству **TimeEnabled** значение **False** в окне **Properties**.

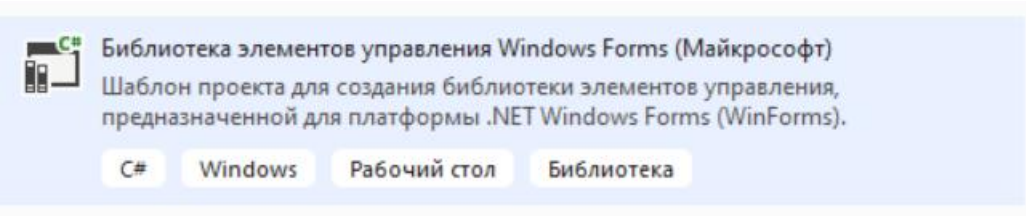
16. Добавьте на форму элемент управления (на ваше усмотрение, например, флажок, кнопку) и реализуйте остановку и запуск таймера, используя его свойство **TimeEnabled**.

17. Постройте и запустите приложение. Обратите внимание, что пользовательский элемент управления во время выполнения действует так же, как в конструкторе.

- **Разработка библиотеки классов элементов управления**

18. Чтобы разработать библиотеку классов элементов управления, выполните следующие действия:

   a. Добавьте в текущее решение новый проект – **Библиотека элементов управления Windows**, назовите, например, WinFormsControlLibraryTimer.



   b. По умолчанию проект содержит составной элемент управления.

   c. Добавьте элементы управления и код, как описано в упражнении 1 до пункта 12.

   d. Создайте библиотеку DLL, построив проект.

- **Применение составного элемента управления**

19. Перенесите из панели **Toolbox** новый компонент **UserControl1** на поверхность формы **Forml** и проверьте его работу аналогично упражнению 1 пункты 14-17.

Упражнение 2. Создание специализированного элемента управления

Для специализированных элементов управления не существует пользовательского интерфейса по умолчанию, они должны предоставлять весь код, необходимый для отображения своего графического представления. Элемент управления разрабатывается для того, чтобы получить точно такое визуальное представление, которого вам необходимо, и закодировать любую требуемую функциональность для взаимодействия с пользователем

Специализированные элементы управления наследуются от класса **Control.** Этот класс обеспечивает функциональность, требуемую для элемента управления. Он предоставляет базовую функциональность, необходимую для взаимодействия элемента управления с остальной частью приложения.

Ключевой задачей при разработке специализированного элемента управления является реализация видимого пользовательского интерфейса. Его можно создать реализацией метода **OnPaint,** вызываемого каждый раз, когда элемент управления отображается на экране.

Выполнив это упражнение, вы научитесь создавать специализированный элемент управления, также представляющий собой цифровые часы. Подобно элементу управления из предыдущего упражнения, они включают компонент **Timer** для обновления пользовательского интерфейса на регулярной основе. Однако теперь вы сами создадите отображение для этого элемента управления вместо того, чтобы использовать элемент управления **Label.**

- **Разработка специализированного элемента управления**

1. Создайте в Visual Studio новое приложение Windows Forms. Назовите его **WinTimer2.**

2. В меню **Project** выберите **Add New Item**. Выберите **Custom Control** (**Настраиваемый элемент управления**) в диалоговом окне **Add New Item** и щелкните **Add**. Укажите имя **UserControlTimer2** и нажмите **Add (Добавить).** К проекту будет добавлен новый специализированный элемент управления. Откройте и просмотрите его код.

3. Откройте новый элемент в режиме конструктора. В этом режиме нет визуального представления как в составном компоненте, но можно добавить требуемые компоненты в класс, перетащив их из панели **Toolbox** – перенесите компонент **Timer** из **Toolbox** на поверхность конструктора.

4. В окне **Properties** присвойте свойству **Interval** компонента **Timer** значение **1000** и свойству **Enabled** значение **True.**

5. Дважды щелкните компонент **Timer** чтобы открыть окно кода в обработчике события **Timer.Tick** по умолчанию и добавьте следующую строку программы:

```
this.Refresh();
```

6. Измените содержимое переопределенного метода **OnPaint()** – укажите код для отображения прямоугольника, залитого зеленым цветом, заполняющего весь элемент управления:

```
protected override void OnPaint(PaintEventArgs pe)
{
      base.OnPaint(pe);
     Graphics g = pe.Graphics;
     g.FillRectangle(Brushes.Green, 0, 0, this.Width, this.Height);
}
```

7. Для отображения времени добавьте следующий код к методу **OnPaint** ниже определения прямоугольника:

```
pe.Graphics.DrawString(DateTime.Now.ToLongTimeString(), this.Font, new
SolidBrush(this.ForeColor), 0, 0);
```

8. Сохраните вннесенные изменения и постройте проект.

- **Применение специализированного элемента управления**

9. Выберите вкладку конструктора **Forml.** Из **Toolbox** перетащите **UserControlTimer2** в форму. К ней будет добавлен экземпляр вашего специализированного элемента управления, который начнет отсчитывать каждую секунду.

10.Постройте и запустите приложение. Обратите внимание, что пользовательский элемент управления действует во время выполнения таким же способом, как в конструкторе.

Упражнение 3. Создание расширенных элементов управления

Расширенные элементы управления — это разработанные пользователем элементы управления, расширяющие уже существующий элемент управления.

Сохраняется вся функциональность существующих элементов управления, но при этом добавляются свойства и методы, а при необходимости изменяется и внешнее представление элемента управления.

➢ **Разработка расширенного элемента управления**

1. Создайте в Visual Studio новое приложение Windows Forms. Назовите его **WinButNum.**

2. В меню **Project** выберите **Add Class**. Назовите этот класс **ClickButton** и щелкните **Add**.

3. Измените объявление класса так, чтобы **ClickButton** наследовал класс **Button**:

```
public class ClickButton : System.Windows.Forms.Button
```

4. Добавьте в класс следующее свойство **MyClicks**:

```
public int MyClicks{ get; private set; }
```

5. Переопределите метод **OnClick**, чтобы инкрементировать свойство **MyClicks** каждый раз, когда щелкается кнопка:

```
protected override void OnClick(EventArgs e)
{
     MyClicks++;
     base.OnClick(e);
}
```

6. Переопределите метод **OnPaint**, чтобы отобразить количество щелчков в правом нижнем углу элемента управления:

```
protected override void OnPaint(PaintEventArgs pevent)
{
     base.OnPaint(pevent);
     System.Drawing.Graphics g = pevent.Graphics;
     System.Drawing.SizeF stringsize;
     stringsize = g.MeasureString(MyClicks.ToString(), this.Font, this.Width);
     g.DrawString(MyClicks.ToString(), this.Font,
     System.Drawing.SystemBrushes.ControlText,
        this.Width - stringsize.Width - 3,
        this.Height - stringsize.Height - 3);
}
```

7. Сохраните и постройте решение.

➢ **Применение расширенного элемента управления**

8. Выберите вкладку конструктора Forml.
9. Из **Toolbox** перетащите экземпляр **ClickButton** в форму и измените его размеры на ваше усмотрение для наглядного отображения.
10. Постройте и запустите приложение.
11. Проверьте работу кнопки – обратите внимание на отображение счетчика щелчков в правом нижнем углу.

Упражнение 4 (контрольное). Форма валидации данных

Реализуйте форму с валидацией вводимых данных в виде пользовательского компонента. Внедрите его в новое приложение, передайте введенные проверенные данные в модель (свойства класса пользователя) и

выведете информацию о пользователе в подходящий для этого элемент управления формы.

Лабораторная работа 4. Использование окон диалога

Классической задачей при работе с приложениями является получение ввода некоторой информации от пользователя. Решается данная задача чаще всего посредством диалоговых окон.

Упражнение 1. Использование стандартных окон диалога

Задание 1. Использование компонента SaveFileDialog

Чтобы пользователи могли сохранять файлы, можно использовать встроенный компонент **SaveFileDialog**.

В этом упражнении вы отобразите диалоговое окно, используя метод **ShowDialog**. Затем с помощью поля **DialogResult.OK** проверите, нажал ли пользователь кнопку **OK**.

Для реализации отображения диалогового окна обозревателя папок выполните:

1. Создайте приложение Windows Forms, укажите имя **TestStandartDialog**.
2. Добавьте элемент **MenuStrip**, задайте имя первого пункта меню **Файл** и команду **Сохранить как…**
3. Добавьте в форму элемент управления **richTextBox**, оставив имя по умолчанию **richTextBox1**. Свойству **Dock** установите **Fill**.
4. Добавьте в форму компонент **SaveFileDialog**. Проверьте, что в области компонентов появился компонент **saveFileDialog1**.
5. Дважды щелкните кнопку, чтобы добавить в редактор кода обработчик событий по умолчанию.
6. В обработчике событий добавьте следующий код для отображения диалогового окна **Сохранение файла**. Этот код сохраняет текст, введенный в элемент управления **richTextBox**, в текстовый файл в указанной папке.

```
saveFileDialog1.Filter = "txt files (*.txt)|*.txt";
if(saveFileDialog1.ShowDialog() == System.Windows.Forms.DialogResult.OK
    && saveFileDialog1.FileName.Length > 0)
{
    richTextBox1.SaveFile(saveFileDialog1.FileName,
        RichTextBoxStreamType.PlainText);
}
```

7. Постройте и протестируйте приложение.
8. В открывшейся форме введите какой-либо текст в текстовое поле.
9. Выберите команду **Сохранить как…** и сохраните файл (имя и место для сохранения файла выберите по своему усмотрению).
10. Убедитесь, что текстовый файл находится в указанном месте.

Задание 2. Использование компонента ColorDialog

Для отображения диалогового окна цветовой палитры можно использовать встроенный компонент **ColorDialog** вместо того, чтобы создавать свое собственное диалоговое окно.

В этом упражнении вы дополните приложение **TestStandartDialog**, чтобы дать пользователям возможность выбрать цвет и применить его в форме Windows после указания соответствующей команды меню.

Для реализации отображения диалогового окна цветовой палитры выполните:

1. Для элемента MenuItem задайте имя второго пункта меню – **Формат** и команду – **Цвет фона**.
2. Добавьте в форму компонент **ColorDialog**.
3. Проверьте, что в области компонентов появился компонент **colorDialog1**.
4. Дважды щелкните кнопку **Цвет фона**, чтобы создать обработчик событий по умолчанию в редакторе кода.
5. В обработчике событий добавьте следующий код для отображения диалогового окна выбора цвета и изменения фонового цвета в соответствии с выбором пользователя:

```
if (colorDialog1.ShowDialog() == DialogResult.OK)
    {
      richTextBox1.BackColor = colorDialog1.Color;
    }
```

6. Постройте и протестируйте приложение.

Задание 3. Использование компонента FontDialog

Для отображения диалогового окна выбора шрифтов можно использовать встроенный компонент **FontDialog** вместо того, чтобы создавать свое собственное диалоговое окно.

В этом упражнении вы дополните приложение **TestStandartDialog**, чтобы дать пользователям возможность выбрать шрифт в диалоговом окне и затем применить его к тексту.

Для реализации отображения диалогового окна выбора шрифтов выполните:

1. Задайте в меню **Формат** новую команду – **Шрифт**.
2. Перетащите в форму компонент **FontDialog**.
3. Проверьте, что в области компонентов появится компонент **fontDialog1**.
4. Дважды щелкните команду **Шрифт**, чтобы создать в редакторе кода обработчик событий по умолчанию.
5. В обработчик событий добавьте следующий код для отображения диалогового окна выбора шрифта текста в окне и изменения шрифта текста в соответствии с выбором пользователя:

```
if (fontDialog1.ShowDialog() == DialogResult.OK)
     {
       richTextBox1.Font = fontDialog1.Font;
     }
```

6. Постройте и протестируйте приложение.

Задание 4. Использование компонента OpenFileDialog

Чтобы пользователи могли выбрать текстовый файл и загрузить его в элемент управления **RichTextBox** в форме Windows Forms, можно использовать компонент **OpenFileDialog**.

В этом упражнении вы дополните приложение **TestStandartDialog**, чтобы дать пользователям возможность открыть текстовый файл.

1. Задайте в меню **Файл** новую команду **Открыть…**
2. Перетащите в форму компонент **OpenFileDialog**.

   В области компонентов появился компонент **openFileDialog1**.
3. Дважды щелкните команду **Открыть…**, чтобы создать в редакторе кода обработчик событий по умолчанию.
4. В обработчик событий добавьте следующий код для отображения диалогового окна открытия файла:

```
Stream myStream = null;
            OpenFileDialog openFileDialog1 = new OpenFileDialog();
            openFileDialog1.InitialDirectory = @"c:\";
    openFileDialog1.Filter = "txt files (*.txt)|*.txt|All files
(*.*)|*.*";
            openFileDialog1.FilterIndex = 2;
            if (openFileDialog1.ShowDialog() == DialogResult.OK)
            {
                try
                {
                    if ((myStream = openFileDialog1.OpenFile()) != null)
                    {
                        using (myStream)
                        {
              richTextBox1.LoadFile(openFileDialog1.FileName,
                RichTextBoxStreamType.PlainText);
                        }
                    }
                }
                catch (Exception ex)
                {
 MessageBox.Show("Error: Could not read file from disk: " + ex.Message);
                }
            }
```

Постройте и протестируйте приложение.

Упражнение 2. Создание пользовательских модальных диалоговых окон

В упражнении вы реализуете возможность добавления и редактирования сотрудников в список, отображаемый на главной форме. Поля ввода для параметров, добавляемого или редактируемого объекта будут расположены в отдельном окне диалога, который вы реализуете.

- **Создание главной формы приложения и окна диалога**

1. Создайте новое Windows-приложение и назовите его **EditPerson**.
2. Добавьте на форму компонент **ListView** для хранения списка и две кнопки для добавления новой записи и редактирования выделенной.
3. В компоненте **ListView** с помощью свойства **Columns** добавьте три колонки с заголовками **Имя**, **Фамилия** и **Возраст**. Свойству **View** установите значение **Details**.
4. Свойству **(Name)** компонента **ListView** установите значение **personsListView**.
5. Для формы установите заголовок **Список сотрудников** и настройте свойства формы так, чтобы нельзя было ее разворачивать и менять размеры (свойство **FormBorderStyle** желательно установить в FixedDialog). Примерно форма может выглядеть как на рис. 2.1.

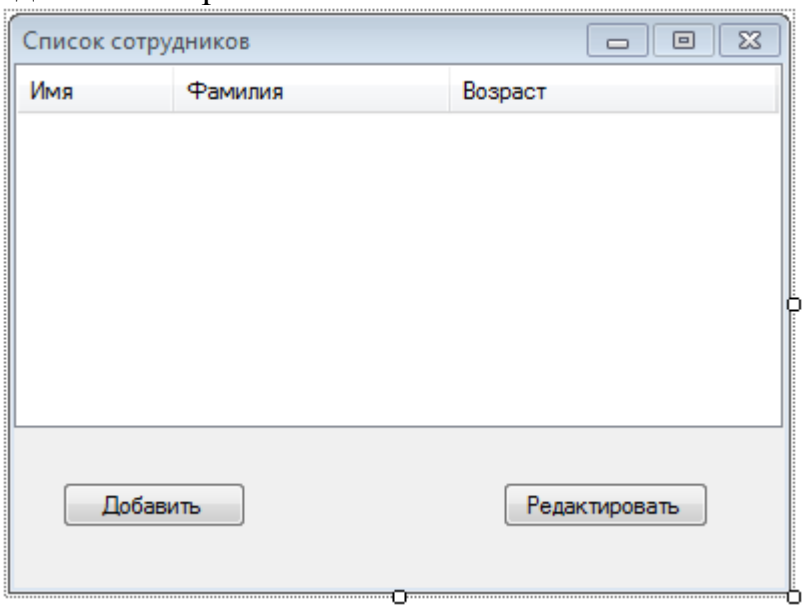


Рис. 2.1. Главная форма

6. Добавьте в проект новую форму – **EditPersonForm.**
7. На форму добавьте текстовые надписи, поясняющие вводимые данные и соответствующие поля ввода (свойство **(Name)**):
   a. Для ввода имени – **firstNameTextBox**;
   b. Для ввода фамилии – **lastNameTextBox**;
   c. Для ввода возраста – **ageNumericUpDown**.
8. Добавьте на форму две кнопки для сохранения и отмены изменений.
9. Для формы установите заголовок **Редактирование сотрудника** и настройте свойства формы так, чтобы нельзя было ее разворачивать и менять размеры.
10. Укажите свойству формы **AcceptButton** кнопку сохранения изменений, а свойство **ShowInTaskBar** установите в **false**, потому

что только главное окно должно отображаться в панели задач. Примерно форма может выглядеть как на рис. 2.2.

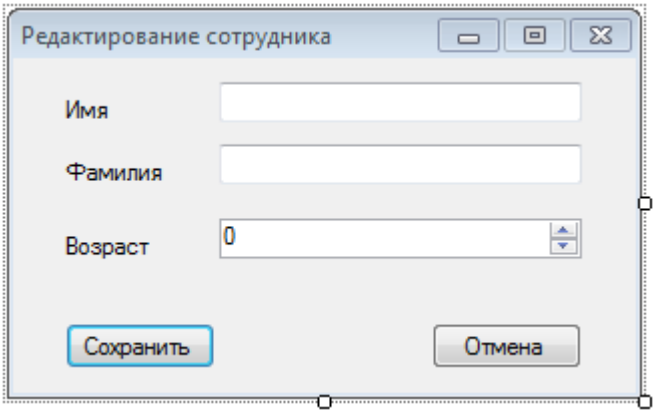


Рис. 2.2 Форма диалогового окна

11. Для кнопки **Сохранить** установите свойство **DialogResult** в OK, а для кнопки **Отмена** установите это же свойство в **Cancel**. В этом случае при отображении окна в модальном режиме по нажатию любой из этих кнопок окно будет закрыто, а форма вернет в качестве результата то, что мы укажем для кнопок в свойстве **DialogResult**.

➢ **Настройка свойств диалогового окна**

1. Добавьте в код формы диалогового окна (класс **EditPersonForm**) свойства с помощью, которых будет реализовано обращение к полям ввода:

```
public string FirstName
{
    get { return firstNameTextBox.Text; }
    set { firstNameTextBox.Text = value; }
}

public string LastName
{
    get { return lastNameTextBox.Text; }
    set { lastNameTextBox.Text = value; }
}

public int Age
{
    get { return (int)ageNumericUpDown.Value; }
    set { ageNumericUpDown.Value = value; }
}
```

➢ **Реализация заполнения главной формы**

1. В класс главной формы добавьте обработчик события **Click** для кнопки **Добавить**.
2. В обработчике события:
   a. Создайте экземпляр формы диалогового окна:

```
EditPersonForm editForm = new EditPersonForm();
```

b. Отобразите форму диалогового окна с помощью метода **ShowDialog()** и если будет нажата в диалоговом окне кнопка OK реализуйте заполнение списка **ListView** главной формы:

```
EditPersonForm editForm = new EditPersonForm();
if (editForm.ShowDialog() != DialogResult.OK)
    return;
ListViewItem newItem =
personsListView.Items.Add(editForm.FirstName);
newItem.SubItems.Add(editForm.LastName);
newItem.SubItems.Add(editForm.Age.ToString());
```

3. В класс главной формы добавьте обработчик события **Click** для кнопки **Редактировать**.
4. В обработчике события:
   a. Вначале метода реализуйте проверку, что не выбран элемент для редактирования и присвойте выбранный элемент экземпляру списка:

```
if (personsListView.SelectedItems.Count == 0)
                return;
ListViewItem item = personsListView.SelectedItems[0];
```

   b. Создайте форму диалогового окна и передайте содержимое элементов списка главной формы в свойства диалогового окна:

```
EditPersonForm editForm = new EditPersonForm();

editForm.FirstName = item.Text;
editForm.LastName = item.SubItems[1].Text;
editForm.Age = Convert.ToInt32(item.SubItems[2].Text);
```

   c. В случае изменения данных и нажатия кнопки **Сохранить** на форме диалога передайте новые значения свойств в соответствующие элементы списка:

```
if (editForm.ShowDialog() != DialogResult.OK)
    return;

item.Text = editForm.FirstName;
item.SubItems[1].Text = editForm.LastName;
item.SubItems[2].Text = editForm.Age.ToString();
```

➢ **Тестирование приложения**

1. Постройте и протестируйте работу приложения. Добавьте несколько сотрудников в список главной формы с помощью формы диалогового окна.
2. Проверьте работу операции редактирования информации о сотруднике – выберите его имя в списке и нажмите **Редактировать**.

Упражнение 3. Взаимодействие компонентов формы и класса приложения

К этому моменту реализовано только добавление данных о сотруднике в список главной формы. В практических задачах обычно требуется реализовать возможность сохранения вводимых данных в свойства некоторого класса, например, **Person** и сохранение информации в коллекцию. В подобных задачах возникает вопрос: как занести данные о сотруднике в коллекцию и в

компонент формы. Если список будет большого объема, то эта операция займет достаточно много времени и, кроме этого, так как имена есть в списке и в представлении одновременно, то это бессмысленное дублирование информации и расход памяти. В случаях, когда уже есть в памяти большой список элементов, лучше переключить представление в виртуальный режим.

- **Создание класса Person**

1. Добавьте в проект новый класс **Person**, моделирующий сущность сотрудника.
2. В классе **Person** объявите автоматические свойства: имя, фамилия и возраст:
```
public string FirstName { get; set; }
public string LastName { get; set; }
public int Age { get; set; }
```
3. Переопределите метод для вывода информации о сотруднике:
```
public override string ToString()
{
 return LastName + " " + FirstName + "\nВозраст: " + Age + "\n";
}
```

- **Включение виртуального режима компонента ListView**

1. Откройте главную форму в режиме конструктора.
2. Для компонента **ListView** включите виртуальный режим: свойству **VirtualMode** установите значение **True**. В виртуальном режиме компонент не использует элементы из **Items**.

- **Использование класса Person**

1. В поле класса главной формы добавьте код для создания коллекции, которая будет хранить объекты класса **Person**:
```
List<Person> pers = new List<Person>();
```
2. В обработчике события `Click` кнопки **Добавить** главной формы класса **Form** добавьте код создающий объект класса **Person** до создания экземпляра формы диалогового окна:
```
Person p = new Person();
```
3. Передайте конструктору формы диалогового окна созданный объект **p**. Ранее создавалась форма конструктором по умолчанию, после этого все поля формы заполнялись через свойства.
```
EditPersonForm editForm = new EditPersonForm(p);
if (editForm.ShowDialog() != DialogResult.OK)
    return;
```
4. Добавьте созданный объект в коллекцию:
```
pers.Add(p);
```

- **Настройка виртуального режима**

Компонент **ListView** работает виртуально и не хранит элементы. Можно только сообщить компоненту, сколько элементов он может отобразить, т. е. сколько элементов есть в списке. Это делается через свойство `VirtualListSize`. А когда компоненту нужно отобразить определенный элемент из списка, он запрашивает его данные с помощью события `RetrieveVirtualItem`.

1. В обработчике события `Click` кнопки **Добавить** главной формы класса **Form** после кода добавления объекта в коллекцию (`pers.Add(p)`) свойству VirtualListSize компонента **ListView** присвойте результат подсчета количества элементов в списке и для отображения изменений вызовите метод Invalidate() для перерисовки компонента:
```
personsListView.VirtualListSize = pers.Count;
personsListView.Invalidate();
```
2. Добавьте в класс главной формы обработчик события RetrieveVirtualItem и реализуйте заполнение компонента как ответ на событие, т.е. по мере необходимости:
```
    private void personsListView_RetrieveVirtualItem(object sender,
RetrieveVirtualItemEventArgs e)
    {
        if (e.ItemIndex >= 0 && e.ItemIndex < pers.Count)
        {
            e.Item = new ListViewItem(pers[e.ItemIndex].FirstName);
            e.Item.SubItems.Add(pers[e.ItemIndex].LastName);
            e.Item.SubItems.Add(pers[e.ItemIndex].Age.ToString());
        }
    }
```
3. Внесите изменения в реализацию редактирования элемента. Обработчик события `Click` кнопки **Редактировать** должен выглядеть следующим образом:
```
    private void button2_Click(object sender, EventArgs e)
    {
        if (personsListView.SelectedIndices.Count == 0)
            return;

        Person p = pers[personsListView.SelectedIndices[0]];

        EditPersonForm editForm = new EditPersonForm(p);
        if (editForm.ShowDialog() == DialogResult.OK)
        {
            personsListView.Invalidate();
        }
    }
```

➢ **Изменение класса диалоговой формы**

1. Откройте класс **EditPersonForm** и в поле класса объявите переменную **p** класса **Person**:
```
Person p;
```
2. В класс добавьте конструктор с параметром и реализуйте заполнение элементов соответствующими значениями свойств объекта:
```
    public EditPersonForm(Person p)
    {
        InitializeComponent();

        this.p = p;
        this.FirstName = p.FirstName;
        this.LastName = p.LastName;
        this.Age = p.Age;
    }
```
3. В код формы диалогового окна добавьте обработчик события **Click** для кнопки **Сохранить** и добавьте код для сохранения данных из

соответствующих полей ввода формы в свойства экземпляра класса **Person**:

```
p.FirstName = this.FirstName;
p.LastName = this.LastName;
p.Age = this.Age;
```

4. Для проверки функционирования сохранения данных о сотруднике в свойствах класса добавьте код, отображающий эти данные:

```
MessageBox.Show("Сотрудник: " + p.ToString());
```

5. Постройте и протестируйте работу приложения. Добавьте сотрудника в список главной формы с помощью формы диалогового окна и проверьте работу метода **MessageBox.Show**.

➢ **Тестирование коллекции**

6. Для проверки коллекции объектов добавьте на главную форму новую кнопку **Просмотреть список** и элемент **RichTextBox** (см. рис. 3.1).

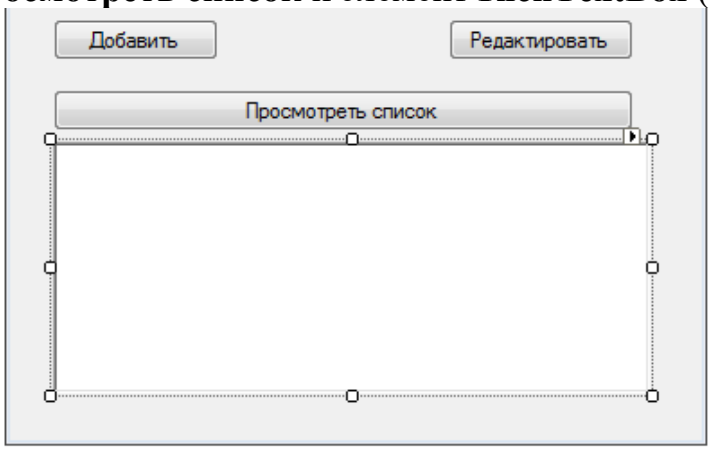


Рис. 3.1. Добавление нового элемента

7. Добавьте обработчик события `Click` кнопки **Просмотреть список** и реализуйте заполнение элемента **RichTextBox** данными о сотрудниках (добавьте `using System.Text;`):

```
private void button3_Click(object sender, EventArgs e)
{
    StringBuilder sb = new StringBuilder();
    foreach (Person item in pers)
    {
      sb.Append("Сотрудник: \n" + item.ToString());
     }
    richTextBox1.Text = sb.ToString();
}
```

8. Постройте и протестируйте работу приложения. Добавьте несколько сотрудников в список главной формы и с помощью формы кнопки **Просмотреть список** проверьте работу коллекции.

Упражнение 4. Создание немодальных окон диалога

В предыдущих упражнениях вы работали с модальными окнами, т. е. с окнами, которые блокируют выполнение родительского окна (из которого модальное окно было вызвано), пока вы не закроете это окно. Немодальные окна не блокируют главное окно, и вы можете работать с двумя окнами одновременно.

Использование свойства Owner

1. Создайте проект *Windows Form* и добавьте на форму два компонента: *TextBox* – в данном компоненте будет выводится информация, передаваемое из второго окна – немодального окна диалога и кнопку *Button* для получения данных из второй формы.
2. В класс формы добавьте свойство с помощью которого будет реализовано обращение к полю вывода (текстовое поле именовано как textBoxF1):

```
public string SolData
{
    get
    { return textBoxF1.Text; }

    set
    { textBoxF1.Text = value; }
}
```

3. Добавьте в проект новую форму.
4. На новой форме разместите три текстовых поля (textBoxF21, textBoxF22 и textBoxF23) и кнопку для передачи данных в первую форму.
5. В обработчике события `Click` кнопки первой формы создайте экземпляр второй формы и в методе `Show()` передайте ей ссылку на первую форму:

```
private void buttonF1_Click(object sender, EventArgs e)
{
    Form2 frm2 = new Form2();
    frm2.Show(this);
}
```

6. В обработчике события `Click` кнопки второй формы объявите строковую переменную, склеивающую три строки, получаемых из текстовых полей, затем объявите переменную первой формы и установите владельца второй формы – ей будет первая форма, и наконец, свойству первой формы (которая определяет доступ к тестовому полю) присвойте объединенную строку, в конце обработчика закройте вторую форму:

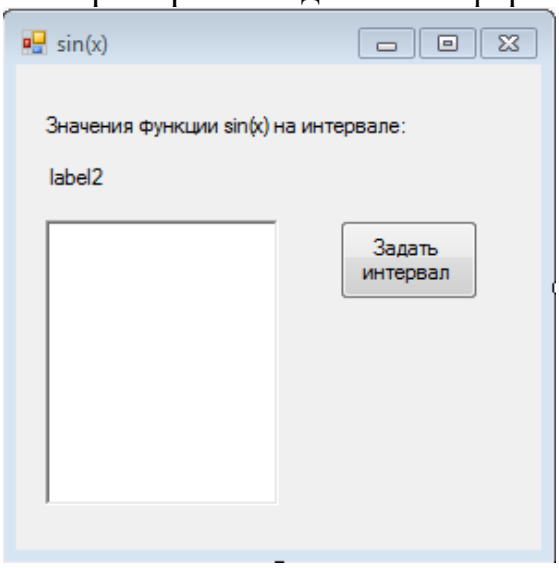
```
private void buttonF2_Click(object sender, EventArgs e)
{
    string s = textBoxF21.Text + " " + textBoxF22.Text + " " +
textBoxF23.Text;
    Form1 frm1 = this.Owner as Form1;
    if (frm1 != null)
    {
        frm1.SolData = s;
    }

    this.Close();
}
```

7. Постройте и протестируйте приложение. Откройте вторую форму, обратите внимание, что она не блокирует работу главного окна. Введите в три текстовых поля информацию и нажмите кнопку. Объединённая строка должна появиться в текстовом поле первой формы.

Упражнение 5 (контрольное). Расчет функции

Требуется реализовать для пользователя возможность ввести данные для расчета функции, например, синуса на заданном интервале. Границы интервала пользователь должен ввести с помощью окна диалога (модального или немодального решите самостоятельно, обоснуйте свой выбор).

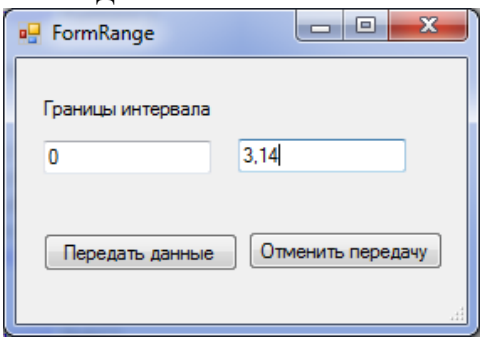
Примерный вид главной формы:



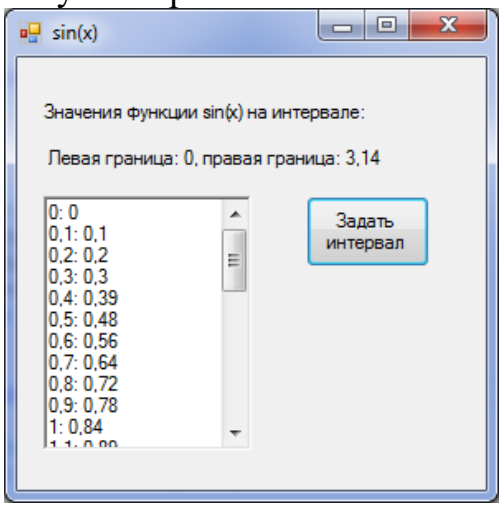
- Диалоговое окно должно открываться по кнопке *Задать интервал*
- Введенные значения интервала должны отобразиться в элементе label2 и использоваться при расчете значений функции
  - Результаты расчетов отображаются в элементе richTextBox

Примерный вид приложения:

Окно диалога



Результат расчета



Лабораторная работа 5. Взаимодействие управляемого и неуправляемого кода

При создании приложений, использующих платформу Microsoft .NET, возникает задача применения в собственных проектах уже готовых библиотек кода, написанных на других языках. Сжатые сроки разработки и уже имеющиеся программные блоки не позволяют отказаться от готовых решений,

поэтому их приходится использовать, встраивая в структуру собственных проектов.

Код, выполняющийся под управлением среды выполнения (в случае платформы .NET — среды Common Language Runtime), называется **управляемым**. Код, запускаемый не под управлением среды, называется **неуправляемым**. Примером неуправляемого кода могут служить COM-компоненты, Microsoft ActiveX интерфейсы и функции API Win32.

Microsoft .NET Framework позволяет взаимодействовать с COM-компонентами, COM+-службами, внешними типами библиотек и разными службами операционной системы.

Платформа .NET Framework предлагает две службы взаимодействия управляемого кода с неуправляемым — Platform Invoke и COM interoperability, которые используют реальные возможности проекта.

Упражнение 1. Получение данных с использованием библиотеки объектов Microsoft Excel

В данном упражнении рассмотрено использование функций приложения Microsoft Office Excel с использованием библиотеки объектов Microsoft Excel. Данная библиотека позволяет управляемому коду взаимодействовать с объектной моделью приложения Microsoft Office, основанной на модели COM.

- **Создание главной формы приложения**

1. Создайте проект *Windows Form* в *Microsoft Visual Studio* и добавьте на форму три компонента:

   textBox1 - в данном компоненте будет выводится путь, имя и расширение выбранного файла;

   button1 – кнопка для запуска процесса получения данных из выбранного файла;

   dataGridView1 – элемент управления необходимый для отображения полученных данных.

2. Выберете компонент **dataGridView1** и откройте окно свойств.
3. Установите следующие параметры в свойствах компонента:

Dock – None – свойство задает границы элемента управления, прикрепленные к его родительскому элементу управления и определяет способ изменения его размеров относительно родительского элемента управления.

Anchor – Top, Bottom, Left, Right – свойство задает границы контейнера, с которым связан элемент управления, и определяет способ изменения его при изменении размеров его родительского элемента.

4. Для кнопки **button1** добавьте обработчик события **Click.**
5. Перейдите в «Обозреватель решений» и найдите группу «**References**» которая содержит все ссылки на внешние компоненты в проекте.

Сделайте клик правой клавишей мыши по данной группе и выберете из появившегося контекстного меню, пункт **Добавить ссылку**…

Откроется окно **Менеджер ссылок**, в левой части данного окна будет предложено выбрать одну из категорий. Visual Studio предоставляет четыре группы для выбора.

Сборки — список всех компонентов платформы .NET Framework, ссылки на которые можно добавить.

Решение — список всех повторно используемых компонентов, созданных в локальных проектах.

COM — список всех COM-компонентов, ссылки на которые можно добавить.

Обзор — позволяет осуществлять поиск компонента в файловой системе.

6. Выберете группу «**COM**» и ее подгруппу «**Библиотеки типов**».

В центральной части окна будет предложен список доступных библиотек для подключения к проекту.

7. Найдите в списке библиотеку «**Microsoft Excel 14.0 Object Library**» (версия может быть другой) и поставьте галочку рядом с именем данной библиотеки. В нижней части окна нажмите кнопку «**ОК**».
8. Проверьте, что после добавления библиотеки в обозревателе решений появились три новых ссылки:

Microsoft.Office.Core;
Microsoft.Office.Interop.Excel;
VBIDE.

9. Для работы с добавленными ссылками добавьте следующие пространства имен:

using System.Reflection;
using ExcelObj = Microsoft.Office.Interop.Excel; - создается псевдоним пространства имен «Microsoft.Office.Interop.Excel».

- **Настройка компонента окна диалога**

1. Перенесите на форму компонент **OpenFileDialog,** реализующий открытие окна диалога для выбора файла.
2. В окне свойств настройте следующие свойства компонента **OpenFileDialog:**

Name = ofd,

DefaultExt = "*.xls;*.xlsx" – задает расширение имени файла по умолчанию,

Title = "Выберите документ для загрузки данных" – задает заголовок диалогового окна.

ofd.Filter = " Excel 2010(*.xlsx)|*.xlsx|Excel 2003(*.xls)|*.xls"; – задает строку фильтра имен файлов, которая определяет варианты, доступные в поле

"Файлы типа" диалогового окна.

После выбора файла будет создаваться новый объект **Application** (приложение **Excel**), которое может содержать одну или более книг, ссылки на которые содержит свойство «**Workbooks**». Книги - объекты «**Workbook**», могут содержать одну или более страниц, ссылки на которые содержит свойство «**Worksheets**». Страницы – «**Worksheet**», могут содержать объекты ячейки или группы ячеек, ссылки на которые становятся доступными через объект «**Range**».

- **Реализация взаимодействия с объектами Excel**

1. В поле класса формы создайте объект приложения и объявите объекты приложения Excel: книга, лист и диапазон:

```
ExcelObj.Application app = new ExcelObj.Application();
ExcelObj.Workbook workbook;
ExcelObj.Worksheet NwSheet;
ExcelObj.Range ShtRange;
```

2. После этого создайте с использованием класса **DataTable** таблицу **dt**, в которую будут заноситься полученные из файла данные:

```
DataTable dt = new DataTable();
```

В следующих пунктах код следует указывать в обработчике события **Click**.

3. Откройте окно диалога и в случае нажатия в нем кнопки OK в элемент управления **textBox1** в текстовое поле с помощью свойства **FileName** объекта **ofd** выведете путь, имя и расширение выбранного файла:

```
if(ofd.ShowDialog() == DialogResult.OK)
{
   textBox1.Text = ofd.FileName;

}
else
{
    MessageBox.Show("Вы не выбрали файл для открытия",
"Загрузка данных...", MessageBoxButtons.OK, MessageBoxIcon.Error);
}
```

В следующих пунктах код следует указывать в теле оператора выбора.

4. Далее для открытия существующего документа примените метод **Open** коллекции **Excel.Workbooks**, в качестве основного параметра укажите путь к файлу, остальные параметры опустите:

```
workbook = app.Workbooks.Open(ofd.FileName);
```

5. Для доступа к листу **NwSheet** из книги **Workbook** примените метод **Sheets.get_Item** с указанием номера листа (учтите, что нумерация листов начинается с 1:

```
NwSheet = (ExcelObj.Worksheet)workbook.Sheets.get_Item(1);
```

6. Чтобы получить объект **ShtRange**, который представляет все ячейки, содержащие значение на данный момент, используете свойство станицы **Worksheet.UsedRange:**

```
ShtRange = NwSheet.UsedRange;
```

7. После получения объекта **ShtRange**, с помощью цикла **For** реализуйте загрузку первой строки из таблицы:

```
            for (int Cnum = 1; Cnum <= ShtRange.Columns.Count; Cnum++)
            {
              dt.Columns.Add(new DataColumn((ShtRange.Cells[1,Cnum] as
ExcelObj.Range).Value2.ToString()));
            }
         dt.AcceptChanges();
```

8. Установите каждое значение первой строки из таблицы в качестве имени колонки таблицы:

```
            string[] columnNames = new String[dt.Columns.Count];
            for (int i = 0; i < dt.Columns.Count; i++)
            {
                columnNames[0] = dt.Columns[i].ColumnName;
            }
```

9. Далее загрузите все оставшиеся строки с добавлением в таблицу:

```
       for (int Rnum = 2; Rnum <= ShtRange.Rows.Count; Rnum++)
       {
           DataRow dr = dt.NewRow();
           for (int Cnum = 1; Cnum <= ShtRange.Columns.Count; Cnum++)
           {
               if ((ShtRange.Cells[Rnum, Cnum] as ExcelObj.Range).Value2
!= null)
               {
                  dr[Cnum - 1] = (ShtRange.Cells[Rnum, Cnum] as
ExcelObj.Range).Value2.ToString();
               }
           }
        dt.Rows.Add(dr);
        dt.AcceptChanges();
       }
```

10. По завершении загрузки данных с указанного листа, сформированную таблицу **dt** подключите к элементу управления **dataGridView1** и закройте объект приложения:

```
            dataGridView1.DataSource = dt;
            app.Quit();
```

➢ **Тестирование приложения**

1. Постройте и запустите приложение. В качестве тестового примера книги Excel укажите файл с таблицей данных, например bb.xlsx.
2. Закройте приложение.

Упражнение 2. Использование COM-компонента для создания PDF-приложения

1. Создайте новое Windows-приложение и назовите его **PDF Reader**.
2. Добавьте на форму элементы **OpenFileDialog** и **MenuStrip**.
3. Установите свойству WindowState формы **Form1**: значение Maximized.
4. Свойство **FileName** элемента **OpenFileDialog** сделайте пустым.

5. Добавьте пункт меню верхнего уровня **File** с командами **Open** и **Exit.**

6. Выберите **Tools | Choose Toolbox Items…**

7. На вкладке **COM Components** выберите **Adobe Acrobat (PDF) Reader** (этот компонент появляется после установки программы Adobe Acrobat Reader**)** и нажмите **OK**. Убедитесь, что на панели Toolbox этот ЭУ действительно появился.

8. Перенесите **Adobe Acrobat Reader** на форму и установите свойству **Dock** значение **Fill**, а свойству **(Name)** – **axAcroPDF1** (от имени объекта будет вызываться нужные нам методы, в принципе имя может быть любым).

9. Добавьте обработчик пункта меню **Open**:

```
private void openToolStripMenuItem_Click (object sender, System.EventArgs e)
     {
     openFileDialog1.Filter = "Файлы pdf|*.pdf";
     openFileDialog1.ShowDialog();
     axAcroPDF1.LoadFile(openFileDialog1.FileName);
     }
```

10. Реализуйте обработчик события **Click** для пункта меню **Exit**.

11. Постройте и запустите приложение. При открытии документа в формате pdf происходит, по сути, встраивание в форму интерфейса программы Adobe Acrobat Reader.

Упражнение 3. Вызов функции API

1. Создайте новое приложение и назовите его WinAnim.

2. Расположите на форме три кнопки (местоположение и размер установите по своему усмотрению) и установите следующие свойства формы и кнопок:

Свойство	**Значение**
Form1	
Text	Анимация формы
Button1	
Name	btnAW_BLEND
Text	Проявление
Button2	
Name	btnHOR_AW_SLIDE
Text	Горизонтальное появление
Button3	
Name	btnCenter_AW_SLIDE
Text	Появление из центра

3. Добавьте класс WinAPIClass:

```
using System;
using System.Runtime.InteropServices;
```

```
using System.Windows.Forms;
namespace AnimatedWindow
{
public class WinAPIClass
{
#region Анимация окна
/// <summary>
/// Тип анимации окна. Перечисление возвращает тип данных int
/// после приведения. Каждому элементу перечисления присвоено
/// свое значение типа int.
/// </summary>
[Flags]
public enum AnimateWindowFlags:int
{
AW_HOR_POSITIVE = 1,
AW_HOR_NEGATIVE = 2,
AW_VER_POSITIVE = 4,
AW_VER_NEGATIVE = 8,
AW_CENTER = 16,
AW_HIDE = 65536,
AW_ACTIVATE = 131072,
AW_SLIDE = 262144,
AW_BLEND = 524288
};
/// <summary>
/// Анимация окна.
/// </summary>
/// <param name="hwnd">Окно.</param>
/// <param name="dwTime">Время.</param>
/// <param name="dwFlags">Тип анимации. Если в неуправляемом
/// коде используется перечисление, то его нужно конвертировать
/// в тип данных int. </param>
/// <returns></returns>
[DllImportAttribute("user32.dll",EntryPoint="AnimateWindow",SetLastError=
true)]
public static extern bool AnimateWindow(IntPtr hwnd,int dwTime,int
dwFlags);
/// <summary>
/// Анимация окна.
/// </summary>
/// <param name="ctrl">Окно.</param>
/// <param name="dwTime">Время.</param>
/// <param name="Flags">Флаги.</param>
/// <returns></returns>
public static bool AnimateWindow(Control ctrl,int dwTime,
AnimateWindowFlags Flags)
{
return AnimateWindow(ctrl.Handle,dwTime,(int)Flags);
}
#endregion
}
}
```

4. Создайте обработчики кнопок:

```
private void btnAW_BLEND_Click(object sender, System.EventArgs e)
{
// Скрываем окно
this.Hide();
// Запускаем анимацию. Второй параметр в скобках — время анимации в
миллисекундах.
WinAPIClass.AnimateWindow(this, 3000,
WinAPIClass.AnimateWindowFlags.AW_ACTIVATE|
WinAPIClass.AnimateWindowFlags.AW_BLEND);
// Отображаем кнопки после анимации
```

```
this.btnAW_BLEND.Invalidate();
this.btnHOR_AW_SLIDE.Invalidate();
this.btnCenter_AW_SLIDE.Invalidate();
}
private void btnHOR_AW_SLIDE_Click(object sender, System.EventArgs e)
{
this.Hide();
WinAPIClass.AnimateWindow(this, 3000,
WinAPIClass.AnimateWindowFlags.AW_HOR_POSITIVE|
WinAPIClass.AnimateWindowFlags.AW_SLIDE);
this.btnAW_BLEND.Invalidate();
this.btnHOR_AW_SLIDE.Invalidate();
this.btnCenter_AW_SLIDE.Invalidate();
}
private void btnCenter_AW_SLIDE_Click(object sender, System.EventArgs e)
{
this.Hide();
WinAPIClass.AnimateWindow(this, 3000,
WinAPIClass.AnimateWindowFlags.AW_CENTER|
WinAPIClass.AnimateWindowFlags.AW_SLIDE);
this.btnAW_BLEND.Invalidate();
this.btnHOR_AW_SLIDE.Invalidate();
this.btnCenter_AW_SLIDE.Invalidate();
}
```

5. Постройте и запустите приложение. Протестируйте три вида анимации.

Упражнение 4. Программное создание документа Microsoft Word

В данном упражнении рассмотрено сохдание приложения Microsoft Office Word с использованием библиотеки объектов Microsoft Word. Данная библиотека позволяет управляемому коду взаимодействовать с объектной моделью приложения Microsoft Office, основанной на модели COM.

- **Создание главной формы приложения**

1. Создайте проект *Windows Form* в *Microsoft Visual Studio* и добавьте на форму кнопку:

   button1 – кнопка для запуска процесса создания документа и вызова MS Word для отображения.

2. Добавьте ссылку на библиотеку «**Microsoft Word 14.0 Object Library**» (версия может быть другой) как это было сделано в первом упражнении с библиотекой Excel.
3. Проверьте, что после добавления библиотеки в обозревателе решений появились три новых ссылки: Microsoft.Office.Core; Microsoft.Office.Interop. Word и VBIDE.
4. Для работы с объектами библиотеки добавьте следующие пространства имен:

```
using System.Reflection;
using WordObj = Microsoft.Office.Interop.Word;
```

Для работы с приложением Microsoft Word в .NET, используется объект **Application**, который является предком всех остальных объектов. Получив на

него ссылку, вы сможете работать с его методами и свойствами. Этот объект предоставляет большой набор методов и свойств, позволяющих программным путем управлять Microsoft Word.

5. В поле класса формы объявите ссылку на объект **Application**:

```
WordObj.Application winword;
```

6. В классе формы создайте метод NewDocument() – в этом методе будет реализована вся работа с документом.
7. В методе реализуйте создание приложения Word – инициализируйте новый объект **Application** и сделайте его невидимым:

```
private void NewDocument()
{
    winword = new WordObj.Application();
    winword.Visible = false;
}
```

8. Чтобы создать новый документ необходимо вызвать метод Add() коллекции объектов **Documents**:

```
WordObj.Document document = winword.Documents.Add();
```

Выполняя какую-либо операцию в пользовательском интерфейсе Word (например, добавляя верхний колонтитул), вы выделяете соответствующую секцию, используя объект **Selection**, определяющий текущую позицию «*невидимого*» курсора и применяете к ней новый параметр форматирования с использованием объекта **Range**. Данный объект представляет область в документе и может включать в себя все что угодно – от пары символов, до таблиц и закладок. В одном документе можно определить сразу несколько таких объектов.

9. Для реализации верхнего колонтитула добавьте следующий код:

```
foreach (WordObj.Section section in document.Sections)
{
   WordObj.Range headerRange =
section.Headers[WordObj.WdHeaderFooterIndex.wdHeaderFooterPrimary].Range;
   headerRange.Fields.Add(headerRange, WordObj.WdFieldType.wdFieldPage);
   headerRange.ParagraphFormat.Alignment =
WordObj.WdParagraphAlignment.wdAlignParagraphCenter;
   headerRange.Font.ColorIndex = WordObj.WdColorIndex.wdBlue;
   headerRange.Font.Size = 10;
   headerRange.Text = "Это верхний колонтитул:" + Environment.NewLine +
"Отчет";
 }
```

10. Для реализации нижнего колонтитула добавьте следующий код:

```
foreach (WordObj.Section wordSection in document.Sections)
{
    WordObj.Range footerRange =
wordSection.Footers[WordObj.WdHeaderFooterIndex.wdHeaderFooterPrimary].Ra
nge;
    footerRange.Font.ColorIndex = WordObj.WdColorIndex.wdDarkRed;
    footerRange.Font.Size = 10;
    footerRange.ParagraphFormat.Alignment =
    WordObj.WdParagraphAlignment.wdAlignParagraphCenter;
    footerRange.Text = "Это нижний колонтитул:" + Environment.NewLine +
"Отчет";
```

```
}
```

11. Чтобы добавить текст в документ укажите позицию для вставки и сам текст:

```
document.Content.SetRange(0, 0);
document.Content.Text = "Отчет" + Environment.NewLine;;
```

12. Добавьте далее текст с определенным стилем:

```
WordObj.Paragraph para1 = document.Content.Paragraphs.Add();
object styleHeading1 = "Заголовок 1";
para1.Range.set_Style(styleHeading1);
para1.Range.Text = "Результаты";
para1.Range.InsertParagraphAfter();
```

В классе **Microsoft.Office.Interop.Word.Document** есть коллекция **Tables**, которая позволяет добавить таблицу в документ с использованием метода **Add**.

13. Добавьте следующий код для создания таблицы. Обратите внимание на комментарии. Текст в шапке таблице демонстрационный, вы можете заменить его на свой, также заполняются и ячейки – в этом примере они просто заполняются цифрами по порядку:

```
//Создание таблицы 5x5
 WordObj.Table firstTable = document.Tables.Add(para1.Range, 5, 5);

 firstTable.Borders.Enable = 1;
 foreach (WordObj.Row row in firstTable.Rows)
 {
     foreach (WordObj.Cell cell in row.Cells)
     {
         //Заголовок таблицы
         if (cell.RowIndex == 1)
         {
           cell.Range.Text = "Колонка " + cell.ColumnIndex.ToString();
           cell.Range.Font.Bold = 1;
               //Задаем шрифт и размер текста
           cell.Range.Font.Name = "verdana";
           cell.Range.Font.Size = 10;
           cell.Shading.BackgroundPatternColor =
WordObj.WdColor.wdColorGray25;
               //Выравнивание текста в заголовках столбцов по центру
           cell.VerticalAlignment =
WordObj.WdCellVerticalAlignment.wdCellAlignVerticalCenter;
           cell.Range.ParagraphFormat.Alignment =
WordObj.WdParagraphAlignment.wdAlignParagraphCenter;
          }
            //Значения ячеек
         else
         {
           cell.Range.Text = (cell.RowIndex - 2 +
cell.ColumnIndex).ToString();
          }
     }
}
```

14. После завершения формирования документа сделайте его видимым:

```
winword.Visible = true;
```

15. Для запуска процесса создания документа добавьте обработчик

события Click и в нем вызовите метод `NewDocument()`:

```
private void button1_Click(object sender, EventArgs e)
{
    try
    {
        NewDocument();
    }
    catch (Exception ex)
    {
        MessageBox.Show(ex.Message);
    }
}
```

16. Постройте и запустите приложение.
17. Нажмите на кнопку, расположенную на форме вашего проекта. Автоматически откроется документ Microsoft Word с заполненным верхним и нижним колонтитулом, обычным и с применением стиля текстом, а также заполненной таблицей.

В данном случае пользователь может внести изменения в созданный документ и потом самостоятельно его сохранить и закрыть.

18. Если требуется, чтобы документ после создания автоматически сохранялся и закрывался, то добавьте следующий код в конце метода NewDocument() (обратите внимание на место сохранения – диск d, при желании можете указать свой путь):

```
//Сохранение документа
object filename = @"d:Demotemp.docx";
document.SaveAs(ref filename);
//Закрытие текущего документа
document.Close();
document = null;
//Закрытие приложения Word
winword.Quit();
winword = null;
```

19. Постройте и запустите приложение. Проверьте, что теперь после создания документа он сохранился в указанном месте под указанным именем.
20. Если не требуется показывать документ после его создания, а нужно его сразу сохранить и закрыть, то удалите строку `winword.Visible = true;`.

Лабораторная работа 6. Графика и анимация в формах

Упражнение 1. Вывод графика функции

В этом упражнении вы реализуете возможность вывода графика функции с помощью интерфейса GDI+. Чтобы нарисовать линию с помощью интерфейса GDI+ вы создадите два объекта: объект **Graphics** и объект **Pen**. Объект **Graphics** содержит методы, непосредственно выполняющие рисование,

а объект **Pen** служит хранилищем атрибутов, таких как цвет, ширина и стиль линии.

➢ **Создание интерфейса приложения**

1. Создайте новое Windows-приложение и назовите его **WinGrafSinLine**.
2. Добавьте на форму контейнерный элемент, например, **GroupBox** и наполните его текстовой информацией, двумя элементами **TextBox** для ввода параметров (функция будет такой: y = sin(a*x) + sin(b*x)) и кнопкой для пересчета как на рис. 1.1.
3. Добавьте на форму панель **Panel**. В этом элементе будет отображаться график функции. Размер панели установите примерно 500;300. При необходимости увеличьте размер формы. Свойству **BorderStyle** установите значение **FixedSingle**.

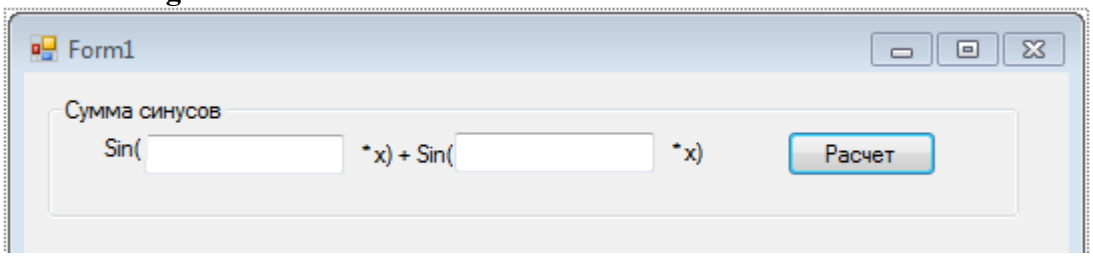


Рис. 1.1. Интерфейс ввода функции

➢ **Создание функции y = sin(a*x) + sin(b*x)**

1. Добавьте в проект новый класс – **Operation**.
2. В классе объявите статический метод, который принимает три параметра: аргумент и два множителя:

```
class Operation
{
    public static double SummSin(int x, double a, double b)
    {
        double res;
        return res = Math.Sin(x * a) + Math.Sin(x * b);
    }
}
```

➢ **Реализация вывода графика функции**

1. В поле класса главной формы объявите две переменных – множители, которые используются в функции и в конструкторе формы присвойте им начальные значения:

```
public partial class Form1 : Form
{
    double a, b;
    public Form1()
    {
        InitializeComponent();
        a = 0.1;
        b = 0.04;
    }
```

2. Создайте два свойства для доступа к текстовым элементам (textBox1 и textBox2) формы:

```
    public string DataA
```

```
        {
            get
            {
                return textBox1.Text;
            }

            set
            {
                textBox1.Text = value;
            }
        }
        public string DataB
        {
            get
            {
                return textBox2.Text;
            }

            set
            {
                textBox2.Text = value;
            }
        }
```

3. Добавьте обработчик события **Paint** панели.

```
private void panel1_Paint(object sender, PaintEventArgs e)
{

}
```

4. В обработчике события **Paint** объявите:
   - переменную – результат функции;
   - две точки – начальную и конечную, график будет строиться короткими отрезками между двумя точками;
   - две переменные, равные ширине и высоте панели, они будут использоваться для вывода графика;
   - переменную, равной половите высоты панели, она будет использоваться для начала рисования графика:

```
double t = 0;
Point p1, p2;
int w = (int)panel1.Width;
int h = (int)panel1.Height;
int y = h/2;
```

5. График строится в цикле с переменной, которая подставляется в функцию и соответственно изменяет координаты точек построения по оси X от нуля до правой границы панели. Далее в обработчике события **Paint** объявите следующий цикл **For**:

```
for (int x = 0; x < w; x++)
{
}
```

6. В цикле
   a. создайте начальную точку:
      ```
      p1 = new Point(x, y);
      ```
   b. вызовите метод **SummSin** класса **Operation** для расчета текущего значения функции:
      ```
      t = Operation.SummSin(x, a, b);
      ```
   c. пересчитайте значение **y** и сместите график для вывода:
      ```
      y = (int)(t * h/5);
      ```

```
y = y + h / 2;
```

d. создайте конечную точку:

```
p2 = new Point(x, y);
```

e. создайте два объекта: объект **Graphics** и объект **Pen:**

```
Graphics dc = e.Graphics;
 Pen p = new Pen(Color.Red, 1);
```

f. вызовите метод рисования линии, передав ему объект-перо и две точки – начальную и конечную).

```
dc.DrawLine(p, p1, p2);
```

7. Постройте и запустите приложение. В панели должен отобразиться график функции с начальными параметрами (см. рис. 2.1).

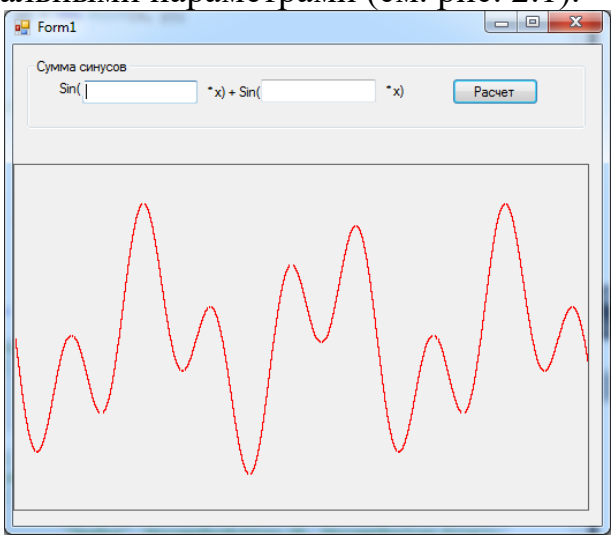


Рис. 2.1. График функции с начальными параметрами

➢ **Реализация ввода новый параметров функции**

1. Добавьте обработчик события **Click** кнопки **Расчет**.
2. В обработчике присвойте ранее объявленным переменным значения, вводимые из элементов **TextBox** и обновите панель для изменения графика. Так как при вводе данных возможен ошибочный ввод этот код поместите в блок **try** и реализуйте обработку ошибок:

```
try
{
    a = double.Parse(DataA);
    b = double.Parse(DataB);
    panel1.Refresh();
}
catch (Exception er)
{
    MessageBox.Show("При выполнении ввода исходных данных
возникла ошибка: \n" + er.Message,"Ошибка", MessageBoxButtons.OK,
MessageBoxIcon.Error);

    return;
}
```

3. Постройте и протестируйте приложение. Введите новые параметры и проверьте, что новый график отобразился на панели. Введите не корректные данные, проверьте работу обработчика ошибок.

➢ **Реализация ввода новых параметров с помощью окна диалога**

В этой части упражнения вы реализуете передачу данных из одной формы (окна немодального диалога) в другую – главную форму, отображающую график функции с помощью свойства **Owner**.

1. Добавьте в проект новую форму (свойству (**Name**) установите значение `edit`), которая будет принимать значение двух параметров и передавать их в первую форму.
2. Разместите на новой форме два элемента **TestBox** и одну кнопку. Текстовое описание элементов реализуйте по своему усмотрению.
3. На главную форму добавьте новую кнопку для открытия второй формы и в обработчике события **Click** этой кнопки создайте экземпляр второй формы, а затем откройте ее передав в метод открытия ссылку на текущую (главную) форму, например:
```
edit editF = new edit();
editF.Show(this);
```
4. В обработчике события **Click** кнопки второй формы сначала установите владельца с помощью свойства **Owner**:
```
Form1 frm1 = this.Owner as Form1;
```
5. В блоке **try** реализуйте считывание значений из текстовых полей второй формы и передаче их в переменные a и b, а также в текстовые поля главной формы:
```
try
{
    frm1.a = double.Parse(textBox1.Text);
    frm1.b = double.Parse(textBox2.Text);
    frm1.DataA = textBox1.Text;
    frm1.DataB = textBox2.Text;
}
catch (Exception er)
{
    MessageBox.Show("При выполнении ввода исходных данных
возникла ошибка: \n" + er.Message,
                "Ошибка", MessageBoxButtons.OK,
MessageBoxIcon.Error);

    return;
}
```
6. Затем закройте вторую форму (текущую) и вызовите метод обновления главной формы для обновления ее панели:
```
this.Close();
frm1.RefreshPanel();
```
7. Но в классе главной формы нет такого метода – для обновления панели вызывался ее встроенный метод, а теперь вследствие невозможности доступа из второй формы к приватной панели первой формы требуется создать новый метод. Перейдите в класс первой формы и объявите в ней новый метод, в котором вызовите обновление панели:
```
public void RefreshPanel()
{
    panel1.Refresh();
}
```

8. Постройте и протестируйте приложение. Откройте второе окно и введите новые параметры, проверьте, что новый график отобразился на панели главной формы.

Упражнение 2. Реализация анимации

В этом упражнении Вы создадите приложение, реализующие анимацию в формах с помощью таймера, перемещая мяч в разных частях клиентской области, имитируя отскок мяча от стен.

➢ **Создание интерфейса приложения**

1. Создайте новое Windows-приложение и назовите его **Balls**.
2. Перенесите с панели инструментов на форму компонент **Timer**.
3. Настройте параметры таймера:
   a. **Enabled** = True
   b. **Interval** = 10

4. В классе главной формы (в поле класса) объявите две переменные типа **Point**, первая определяет начальную позицию, вторая – приращение координат при перемещении:

```
public partial class Form1 : Form
{
   Point location = new Point(50, 50);
   Point speed = new Point(5, 5);
```

5. В классе формы создайте закрытый метод, реализующий перемещение фигуры и отскок от стен:

```
private void Go()
{
   location = new Point(location.X + speed.X, location.Y + speed.Y);

    if (location.X + 0 > ClientRectangle.Width || location.X < 0)
            speed = new Point(-speed.X, speed.Y);
    if (location.Y + 10 > ClientRectangle.Height || location.Y < 0)
            speed = new Point(speed.X, -speed.Y);
    }
```

6. В классе формы создайте закрытый метод, реализующий графическую фигуру и вызывающий метод перемещения:

```
private void Ball()
{
    Graphics gr = Graphics.FromHwnd(this.Handle);

    gr.FillEllipse(Brushes.Cyan, location.X, location.Y, 10, 10);
    Go();
    }
```

7. Перейдите в режим конструктора формы, выделите компонент **timer1** и с помощью окна **Свойства** и добавьте обработчик события **Tick** для таймера. В классе формы появится заглушка обработчика:

```
private void timer1_Tick(object sender, EventArgs e)
{
```

8. В обработчик добавьте метод обновления таймера и вызовите метод, создающий фигуру:

```
private void timer1_Tick(object sender, EventArgs e)
{
    Refresh();
    Ball();
```

```
}
```

9. Постройте и запустите приложение. Мячик должен перемещаться в рамках формы и отскакивать от стен.

10. Добавьте на форму в верхний угол элемент **CheckBox**. Свойству **Text** задайте значение **Go**. С помощью этого элемента будем включать и останавливать таймер.

11. Добавьте обработчик события, возникающее при каждом изменении свойства Check и реализуйте включение и выключение таймера в зависимости от значения флажка:

```
private void checkBox1_CheckedChanged(object sender, EventArgs e)
{
    if (checkBox1.Checked)
        timer1.Enabled = true;
    else
        timer1.Enabled = false;
}
```

12. Добавьте обработчик события загрузки формы и добавьте тот же код управления таймером в зависимости от значения флажка.

```
private void Form1_Load(object sender, EventArgs e)
{
    if (checkBox1.Checked)
        timer1.Enabled = true;
    else
        timer1.Enabled = false;
}
```

13. Постройте и запустите приложение. Так как начальное значение флажка – выключен, то мячик не должен отображаться на форме. Включите флажок и мячик начнет двигаться. Выключите флажок, таймер остановиться и соответственно остановится мячик.

Упражнение 3. Использование компонента Chart

В этом упражнении Вы примените для отображения данных специальный компонент **Chart**, который предоставляет все свойства, методы и события диаграммы.

- **Создание интерфейса приложения**

1. Создайте новое приложение Windows Forms. назовите его **WindowsFormsChart**.

2. Добавьте в свой проект пакет NuGet **WinForms.DataVisualization**.

После установки элемент управления **Chart** становится доступен на панели инструментов. Вы можете перетащить его на форму и настроить его свойства (ряды, области диаграммы, оси и т. д.) для отображения различных типов диаграмм (столбчатых, линейных, круговых и т. д.) с вашими данными.

3. Добавьте на форму панель **Panel** и поместите в эту панель элемент управления графиками **Chart**.

4. Добавьте на форму контейнерный элемент **GroupBox** и разместите в нем два элемента **RadioButton** – Влажность и Температура.

- **Моделирование данных**

Добавьте класс – **dat**. В этом классе моделируется получение данных для отображения в компоненте **Chart:**

```
class dat
{
    Random r;
    public dat()
    {
        r = new Random();
    }
   public dat(int t)
   {
         r = new Random(t);
   }

   public int random(int t)
   {

         int res = r.Next(t);

         return res;
   }
}
```

- **Настройка компонента Chart**

1. В поле класса главной формы объявите статическую переменную, инициализируйте значением 10 и создайте объект данных, передав в конструктор эту переменную:

```
public partial class Form1 : Form
{
    static int k = 10;
    dat d1 = new dat(k);
```

2. В конструкторе главной формы настройте компонент:
   a. Сначала в цикле для объекта **Series1** коллекции **Series** вызовите метод **AddXY** (позволяет добавить точку с двумя заданными координатами):

```
public Form1()
 {
     InitializeComponent();

     for (int i = 1; i <= k; i++)
     {
         chart1.Series["Series1"].Points.AddXY(i, d1.random(10));
     }
```

   b. Настройте отображение кривой, установив тип кривой – Spline (округленная линия):

```
chart1.Series["Series1"].ChartType = SeriesChartType.Spline;
```

   c. Установите видимость значений на графике в местах перелома кривой и настройте легенду диаграммы:

```
chart1.Series["Series1"].IsValueShownAsLabel = true;
chart1.Series["Series1"].LegendText = "Ветер м/с";
```

3. Постройте и запустите приложение. Проверьте отображение данных о скорости ветра.

➢ **Реализация выбора отображаемых данных**

1. В класс главной формы добавьте метод, принимающий два параметра –максимальное значение случайного числа и подпись ряда в легенде:

```
private void chartPlot(int d, string leg)
{
    chart1.Series["Series1"].Points.Clear();
    for (int i = 1; i <= k; i++)
    {
        chart1.Series["Series1"].Points.AddXY(i, d1.random(d));
    }

    chart1.Series["Series1"].ChartType = SeriesChartType.Spline;
    chart1.Series["Series1"].IsValueShownAsLabel = true;
    chart1.Series["Series1"].LegendText = leg;
}
```

2. Реализуйте события выбора переключателей для отображения данных влажности и температуры вызовом метода с требуемыми параметрами:

```
private void radioButton1_CheckedChanged(object sender, EventArgs e)
{

    if (radioButton1.Checked == true)
    {
        chartPlot(100, "Влажность");
    }

}

private void radioButton2_CheckedChanged(object sender, EventArgs e)
{
    if (radioButton2.Checked == true)
    {
        chartPlot(30, "Температура");
    }

}
```

3. Постройте и запустите приложение. Проверьте отображение данных.
4. Замените код конструктора на вызов метода **chartPlot**. Постройте и проверьте приложение.
5. Добавьте на форму элемент **CheckBox**.
6. Замените в методе **chartPlot** установку типа кривой **Spline** на код, проверяющий состояние флажка и меняющий тип кривой **Spline** на **Column:**

```
if (checkBox1.Checked == true)
   chart1.Series["Series1"].ChartType = SeriesChartType.Column;
else
    chart1.Series["Series1"].ChartType = SeriesChartType.Spline;
```

7. Постройте и запустите приложение. Проверьте отображение данных.

Лабораторная работа 7. Асинхронное программирование

Упражнение 1. Работа с компонентом BackgroundWorker

Выполнив это упражнение, вы научитесь применять компонент **BackgroundWorker**. Вам предстоит добавить к приложению компонент **BackgroundWorker** и написать отнимающий продолжительное время метод, который будет выполняться в отдельном потоке. Затем вы сообщите о

продвижении потока и реализуете функциональность отмены фонового процесса.

1. Создайте новый проект Windows Forms. Назовите его WinBackgroundWorker.

2. Добавьте на форму элементы управления: два элемента **Label**, **TextBox**, **ProgressBar** и две кнопки **Button**.

3. Расположите элементы примерно в соответствии с рис. 1.1:

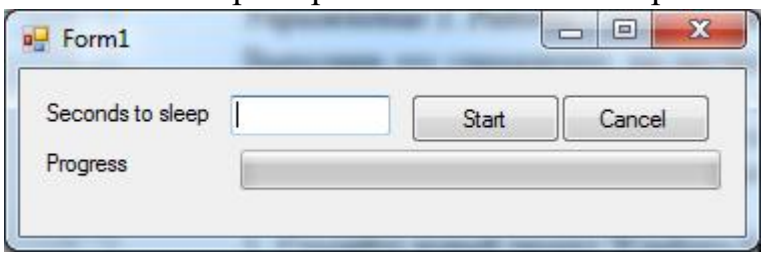


Рис. 1.1 Готовое приложение

4. Для элемента **textBox1** допустимыми значениями будут только цифры, поэтому в обработчике события **KeyPress** для текстового поля **textBox1** укажите код:

```
if (!char.IsDigit(e.KeyChar))
   {
     e.Handled = true;
      MessageBox.Show("Поле должно содержать цифры");
   }
```

5. Из Toolbox перетащите элемент **BackgroundWorker** в форму.

6. В окне Properties установите свойства **WorkerSupportsCancellation** и **WorkerReportsProgress** в *True* (для поддержки асинхронной отмены и возможности сообщения основному потоку информации о продвижении фонового процесса соответственно).

7. Дважды щелкните **BackgroundWorker**, чтобы открыть обработчик события `backgroundWorker1_DoWork` по умолчанию. Добавьте к этому обработчику события следующий код:

```
int i;
i = int.Parse(e.Argument.ToString());
for (int j=1; j <= i; j++)
{
if (backgroundWorker1.CancellationPending)
{
e.Cancel = true;
    return;
}
System.Threading.Thread.Sleep(1000);
    backgroundWorker1.ReportProgress((int)(j *100/ i));
}
```

8. Для элемента **backgroundWorkerl** в окне Properties щелкните кнопку Events, затем дважды щелкните *ProgressChanged,* чтобы открыть окно кода обработчика события `backgroundWorker1_ProgressChanged`. Добавьте следующий код:

```
progressBar1.Value = e.ProgressPercentage;
```

9. Аналогичным способом добавьте обработчик события *RunWorkerCompleted,* в теле обработчика добавьте следующий код:

```
if (!(e.Cancelled))
   System.Windows.Forms.MessageBox.Show("Run Completed!");
    else
System.Windows.Forms.MessageBox.Show("Run Cancelled");
```

10.Для кнопки **Start** откройте обработчик события *Click*. Добавьте следующий код:

```
if (!(textBox1.Text == ""))
    {
int i = int.Parse(textBox1.Text);
backgroundWorker1.RunWorkerAsync(i);
    }
```

11.Для кнопки **Cancel** откройте обработчик события *Click*. Добавьте следующий код

```
  backgroundWorker1.CancelAsync();
```

12.Постройте и выполните приложение, и проверьте его функциональность.

Упражнение 2. Использование делегатов

Выполнив это упражнение, вы создадите приложение, подобное тому, что было создано в упражнении 1. Оно будет выполнять продолжительную операцию в отдельном потоке с возможностью отмены, показывать продвижение операции и уведомлять пользователя о ее завершении. Для реализации этой функциональности вы используете делегаты и асинхронный вызов.

1. Создайте новое Windows-приложение и назовите его WinAsynchDelegate.
2. Повторите шаги 2 – 4 предыдущего упражнения для создания требуемой формы.
3. Добавьте в приложение следующий метод:

```
private void TimeConsumingMethod(int seconds)
        {
            for (int j = 1; j <= seconds; j++)
             System.Threading.Thread.Sleep(1000);
            }
```

4. Используйте в приложении делегат, соответствующий методу *TimeConsumingMethod:*

```
private delegate void TimeConsumingMethodDelegate(int seconds);
```

5. Добавьте сообщающий о продвижении операции метод, который устанавливает значение элемента управления *ProgressBar* потокобезопасным способом, и делегата этого метода:

```
public delegate void SetProgressDelegate(int val);
        public void SetProgress(int val)
        {
            if (progressBar1.InvokeRequired)
            {
       SetProgressDelegate del = new SetProgressDelegate(SetProgress);
                this.Invoke(del, new object[] { val });
            }
            else
            {
                progressBar1.Value = val;
            }
        }
```

6. Для сообщения о продвижении операции добавьте следующую строку кода в цикл *For* метода *TimeConsumingMethod:*

```
SetProgress((int)(j * 100) / seconds);
```

7. Добавьте в приложение логическую переменную с именем *Cancel*:

```
bool Cancel;
```

8. Добавьте следующие строки кода в цикл *For* метода *TimeConsumingMethod:*

```
 if (Cancel)
    break;
```

9. Добавьте следующий код после цикла *For* метода *TimeConsumingMethod:*

```
 if (Cancel)
 {
     System.Windows.Forms.MessageBox.Show("Cancelled");
     Cancel = false;
 }
 else
 {
     System.Windows.Forms.MessageBox.Show("Complete");
      }
```

10. В конструкторе дважды щелкните кнопку Start, чтобы открыть для нее обработчик события *Click* по умолчанию, и добавьте следующий код:

```
TimeConsumingMethodDelegate del = new
TimeConsumingMethodDelegate(TimeConsumingMethod);
        del.BeginInvoke(int.Parse(textBox1.Text), null, null);
```

11. В конструкторе дважды щелкните кнопку Cancel, чтобы открыть для нее обработчик события *Click* по умолчанию, и добавьте следующий код:

```
Cancel = true;
```

12. Скомпилируйте и проверьте ваше приложение.

Упражнение 3. Асинхронный запуск произвольного метода

Хотя компонент **BackgroundWorker** обеспечивает удобный способ выполнения простых задач в фоновом потоке, иногда может потребоваться осуществить более тонкий контроль за фоновыми процессами. В этом упражнении вы научитесь асинхронно выполнять методы с использованием делегатов.

1. Создайте новое Windows-приложение и назовите его WinAsynchMethod.

2. Установите свойствам формы **Size** значение 425;200 и **Text** – "Асинхронный запуск".

3. Добавьте на форму три надписи, два текстовых поля и две кнопки, и расположите их согласно рис. 3.1:

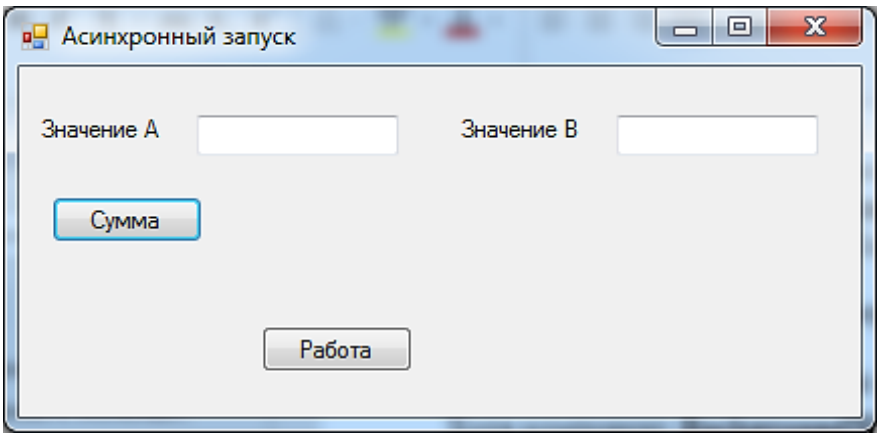


Рис. 3.1 Готовое приложение

4. Установите им следующие свойства:

Свойство	**Значение**
button1	
Name	btnRun
Text	Сумма
button2	
Name	btnWork
Text	Работа
label1	
Name	lblA
Text	Значение A
label2	
Name	lblB
Text	Значение B
textBox1	
Name	txbA
Text	
textBox2	
Name	txbB
Text	

5. Создайте делегат:

```
private delegate int AsyncSumm(int a, int b);
```

6. Создайте метод `Summ`, в котором будут складываться числа, вводимые в два текстовых поля, и укажите задержку операции на 9 секунд:

```
private int Summ(int a, int b)
{
System.Threading.Thread.Sleep(9000);
return a+b;
}
```

7. Реализуйте обработчик кнопки **btnRun**, который будет включать также действия по организации асинхронного вызова:

Создайте экземпляр делегата и проинициализируйте его методом `Summ`:

```
AsyncSumm summdelegate = new AsyncSumm(Summ);
```

Для использования механизма **Callback** создайте экземпляр делегата `AsyncCallBack`:

```
AsyncCallback cb = new AsyncCallback(CallBackMethod);
```

После того как делегат инициализирован методом, можно запускать прикрепленный к делегату метод асинхронно с помощью метода **BeginInvoke**. Этот метод принимает две переменные типа int a и b, экземпляр cb делегата AsyncCallback и экземпляр summdelegate делегата SummDelegate:

```
summdelegate.BeginInvoke(a, b, cb, summdelegate);
```

8. В итоге обработчик кнопки **btnRun** будет выглядеть следующим образом:

```
private void btnRun_Click(object sender, System.EventArgs e)
 {
       int a, b;
 try
       {
             // Преобразование типов данных.
       a = Int32.Parse(txbA.Text);
       b = Int32.Parse(txbB.Text);
       }
 catch(Exception)
       {
 MessageBox.Show("При выполнении преобразования типов возникла ошибка");
       txbA.Text = txbB.Text = "";
 return;
       }
       AsyncSumm summdelegate = new AsyncSumm(Summ);
       AsyncCallback cb = new AsyncCallback(CallBackMethod);
       summdelegate.BeginInvoke(a, b, cb, summdelegate);
       }
```

9. Создайте метод CallBackMethod, который привязан к делегату summdelegate:

```
private void CallBackMethod(IAsyncResult ar)
       {
       string str;
             AsyncSumm summdelegate = (AsyncSumm)ar.AsyncState;
       str = String.Format("Сумма введенных чисел равна {0}",
summdelegate.EndInvoke(ar));
       MessageBox.Show(str, "Результат операции");
       }
```

10.Для демонстрации асинхронности выполнения метода реализуйте обработчик нажатия кнопки **Работа**, например, следующим образом:

```
MessageBox.Show("Работа кипит!!!");
```

11.Постройте и запустите приложение. После нажатия кнопки **Сумма,** пока будет выполняться операция, нажмите кнопку **Работа**. Проверьте, что метод суммирования действительно реализован асинхронно.

Реализация доступа к элементам управления из другого потока

Результат суммирования выводится в окно сообщения. Обычно результаты операций выводятся в элементы управления формы, например, текстовые поля или надписи.

12. Добавьте справа на форму элемент управления надпись и укажите ему имя `lblResult`.

К элементам управления нельзя обращаться из другого потока, и поскольку элементы обычно создаются на форме в основном потоке, а доступ к нему происходит из другого потока, то необходимо использовать метод `Invoke()`.

Метод `Invoke()` компонента вызывает указанный метод в потоке, которому и принадлежит этот компонент. Этот метод возвращает значение, которое возвращает вызываемый делегат, или null, если делегат не возвращает никаких значений.

13. Для вывода результата расчетов объявите в классе формы функцию, которая выводит в компонент, переданный в качестве параметра текст:

```
void PrintFunc(string str)
{
   lblResult.Text = str;
}
```

14. Для вызова этого метода объявите соответствующий делегат и переменную делегата:

```
 delegate void PrintLabel(string str);
 private PrintLabel PrintDlegateFunc;
```

15. В конструкторе формы проинициализируйте переменную делегата (после метода инициализации компонентов):

```
PrintDlegateFunc = new PrintLabel(PrintFunc);
```

16. Вызовите этот метод через делегат с помощью метода Invoke() компонента:

```
blResult.Invoke(PrintDlegateFunc, new object[] { str });
```

Методу `Invoke()` передается два параметра: делегат, который нужно выполнить и массив объектов параметров.

В нашем случае методу делегата нужно передать только один параметр – строку, которая должна выводить результат функции в компоненте.

17. Постройте и запустите приложение. Проверьте, что результат операции выводится в текстовый элемент.

Упражнение 4. Асинхронное программирование с использованием ключевых слов async и await

В этом упражнении вы используете ключевые слова **async** и **await**, которые вместе с функциональностью задач **Task** составляют основу модели асинхронного программирования в .NET, которая называется Task-based Asynchronous Pattern. Данная модель появилась в .NET Framework 4.5.

Операторы **async** и **await** используются вместе для создания асинхронного метода. Такой метод, определенный с помощью модификатора async и содержащий одно или несколько выражений await, называется асинхронным методом.

Ключевое слово **async** указывает, что метод или лямбда-выражение являются асинхронными.

Оператор **await** применяется к задаче в асинхронных методах, чтобы приостановить выполнение метода до тех пор, пока эта задача не завершится. При этом выполнение потока, в котором был вызван асинхронный метод, не прерывается.

1. Откройте приложение WinAsynchMethod, созданное в прошлом упражнении.

2. Добавьте на форму новую кнопку, свойству **Text** установите значение «Разность».

3. Добавьте обработчик события `Click` новой кнопки и укажите ключевое слово **async** в заголовке метода-обработчика события:

```
private async void button2_Click(object sender, EventArgs e)
{
}
```

4. В новый обработчик добавьте код считывания вводимых значений из текстовых полей:

```
int a, b;
try
{
    a = Int32.Parse(txbA.Text);
    b = Int32.Parse(txbB.Text);
}
catch (Exception)
{
    MessageBox.Show("При выполнении преобразования типов
возникла ошибка");
    txbA.Text = txbB.Text = "";
    return;
}
```

5. Далее в обработчике добавьте вызов метода расчета разности с ключевым словом **await** (метод будут вами реализован ниже) и выведете результат в текстовое поле элемента `lblResult`:

```
int res = await Subb(a, b);
lblResult.Text = res.ToString();
```

6. После метода обработчика события определите метод, возвращающий объект **Task** – задачу, которая выполняется асинхронно:

```
private async Task<int> Subb(int a, int b)
{
    return await Task.Run(() =>
    {
        Thread.Sleep(5000);
        return a - b;
    }
    );
}
```

7. Постройте и запустите приложение. Проверьте, что результат операции вычитания выводится в текстовый элемент.

Упражнение 5 (контрольное). Реализация алгоритма во вторичном потоке

Разработайте приложение, в котором реализуйте вывод в подходящий для этого компонент последовательность чисел по алгоритму Эратосфена,

рассчитанных во вторичном потоке. Исходной информацией является передаваемое, например из текстового поля максимальное число последовательности.

В качестве реализации указанного алгоритма можно использовать метод, в котором последовательность до максимального числа (получаемого из текстового поля `MaxValue.Text`) формируется и возвращается в виде строки:

```
public string GoButt()
{
    int maxValue = 0;
    System.Text.StringBuilder resultText = new
 System.Text.StringBuilder();
    if (int.TryParse(MaxValue.Text, out maxValue))
    {
        for (int trial = 2; trial <= maxValue; trial++)
        {
            bool isPrime = true;
            for (int divisor = 2; divisor <= Math.Sqrt(trial); divisor++)
            {
                if (trial % divisor == 0)
                {
                    isPrime = false;
                    break;
                }
            }
            if (isPrime)
            {
                resultText.AppendFormat("{0} ", trial);
            }
        }
    }
    else
    {
        resultText.Append("Unable to parse maximum value.");
    }
    return resultText.ToString();
}
```

Лабораторная работа 8. Повышение удобства использования приложений

Упражнение 1. Использование сериализации для сохранения коллекции объектов

В этом упражнении вы добавите в приложение возможность сохранять информацию о сотрудниках, представленных в виде коллекции с помощью механизма сериализации.

Сериализация описывает процесс сохранения состояния объекта в потоке (например, файловом потоке и потоке в памяти). Последовательность сохраняемых данных содержит всю информацию, необходимую для реконструкции (или десериализации) состояния объекта с целью последующего использования. Применяя эту технологию, очень просто сохранять большие объемы данных в различных форматах с минимальными усилиями. Объект сохраняет все необходимые свойства так, чтобы при загрузке их из потока можно было восстановить работоспособность объекта,

поэтому сохранив состояние объекта, можно выключить программу, а при повторном запуске программы восстановить состояние.

Во многих случаях сохранение данных приложения с использованием служб сериализации удобнее, чем применение классов для чтения/записи из пространства имен System.IO.

- **Сериализация объектов с использованием BinaryFormatter**

Тип **BinaryFormatter** сериализирует состояние объекта в поток, используя компактный двоичный формат. Этот тип определен в пространстве имен System.Runtime.Serialization.Formatters.Binary.

1. Откройте приложение **EditPerson**.
2. Для того чтобы сохранять состояние объекта в потоке **stream** с помощью сериализации перед объявлением класса **Person** добавьте атрибут [**serializable**]:

```
[Serializable]
  public  class Person
    {…
```

3. Сохранение данных будет происходить перед закрытием приложения, поэтому добавьте для главной формы обработчик события **FormClosing**.
4. В обработчике события реализуйте возможность сериализации коллекции **pers**, сначала создайте экземпляр класса **BinaryFormatter**, затем создайте поток, который создает новый файл и открывает его для записи в него данных, и наконец, вызовите метод **Serialize (**методу передайте два параметра: поток, в котором будет происходить сохранение, и объект, который нужно сохранить в потоке – коллекцию **pers**):

```
            private void Form1_FormClosing(object sender,
FormClosingEventArgs e)
        {
            BinaryFormatter binFormat = new BinaryFormatter();
            using (FileStream fStream = new FileStream("AllMyPerson.dat",
              FileMode.Create, FileAccess.Write, FileShare.None))
            {
                binFormat.Serialize(fStream, pers);
            }
        }
```

5. Реализуйте обратную операцию – десериализацию в обработчике события загрузки формы:

```
 private void Form1_Load(object sender, EventArgs e)
 {
            BinaryFormatter binFormat = new BinaryFormatter();
            try
            {

using (FileStream fStream = new FileStream("AllMyPerson.dat",
         FileMode.OpenOrCreate, FileAccess.Read, FileShare.None))
{
   pers.AddRange((List<Person>)binFormat.Deserialize(fStream));
}
            }

            catch
            { }
}
```

6. Постройте и протестируйте приложение. Добавьте несколько записей, закройте приложение и проверьте сохранились ли данные.

Замечание. Начиная с .NET 5 **BinaryFormatter** объявлен устаревшим из-за небезопасности и во время компиляции приводит к созданию предупреждения с идентификатором **SYSLIB0011,** а начиная с .NET 7 – создает ошибку во время компиляции для всех типов проектов C# с тем же идентификатором SYSLIB0011.

Пути решения проблемы

1. Лучше отказаться от BinaryFormatter и перейти на другой метод сериализации и десериализации, более новый/безопасный. Но при переходе на новый формат совместимости со старыми данными не будет.

2. Если все же нужно сохранить BinaryFormatter (данные никуда не передаются/какой-то внутренний формат, т.е. особых требований по безопасности нет), то в файл проекта нужно включить:

```
<EnableUnsafeBinaryFormatterSerialization>true</EnableUnsafeBinaryFormatterSe
    rialization>
```

тогда уйдет сообщение об ошибке, но появится предупреждение, а чтобы и его не было, то следует добавить далее в файл проекта

```
<!-- Disable "BinaryFormatter is obsolete" warnings for entire project -->
<NoWarn>$(NoWarn);SYSLIB0011</NoWarn>
```

В .NET предлагается несколько встроенных сериализаторов, которые обеспечивают безопасную работу с ненадежными данными, один из них XmlSerializer, реализующий сериализацию графов объектов в XML и обратно.

- **Сериализация объектов с использованием XmlSerializer**

Тип **XmlSerializer** сериализирует состояние объекта в документе XML. Этот тип определен в пространстве имен System.Xml.Serialization.

1. Для того чтобы сохранять состояние объекта в документе XML с помощью сериализации в атрибут **Serializable** добавьте атрибут параметр, определяющий корневой узел:

```
     [Serializable,
 XmlRoot(Namespace = "http://www.MyCompany.com")]
```

2. В обработчик закрытия формы добавьте код, реализующий сохранение в XML-файл:

```
    using (FileStream fStream = new FileStream("PersonCollection.xml",
FileMode.Create, FileAccess.Write, FileShare.None))
    {
                XmlSerializer xmlFormat = new
XmlSerializer(typeof(List<Person>));
                xmlFormat.Serialize(fStream, pers);
    }
```

3. Постройте и протестируйте приложение. Добавьте несколько записей, закройте приложение и проверьте содержимое файла PersonCollection.xml в папке Debug.

➢ **Запрещение сериализации для конкретного свойства**

В классе **Person** значение поля **Age** через некоторое время станет неактуальным, поэтому подобные поля сохранять не рекомендуется, и они должны вводиться вручную или рассчитываться автоматически.

Для BinaryFormatter

1. Замените в классе объявления **Person** автоматического свойства **Age** на явное объявление закрытого поля **age** и соответствующего свойства и пометьте поле атрибутом `[Nonserialized]`:

```
[NonSerialized]
int age;
public int Age
{
    get { return age; }
    set { age = value; }
}
```

2. Постройте и протестируйте приложение. Добавьте несколько записей, закройте приложение и проверьте, что значение свойства **Age** при повторном запуске приложения не сохраняется.

Для XmlSerializer

3. Для того, чтобы при XML-сериализации игнорировать свойство, следует применить специальный для этого метод **ShouldSerialize** с определенным суффиксом – именем XML-элемента/свойства (точно такое же) и в методе вернуть логическое значение на основе вычислений или проверки, которую необходимо выполнить, чтобы решить, будет ли элемент нулевым или пустым и включать ли его в XML или нет:

```
public bool ShouldSerializeAge()
{
    return !Convert.ToBoolean(Age);
}
```

4. Проверьте, что теперь атрибута **Age** нет в файле XM – просмотрите содержимое файла PersonCollection.xml.

Реализация средств помощи при работе с программами

В приложениях, содержащих панели инструментов с большим количеством свойств и настроек, принято добавлять средства для повышения удобства работы и практичности, в частности, контекстную справку, всплывающие подсказки, а также файлы со справочной информацией.

Упражнение 2. Создание контекстной справки

1. Откройте Windows-приложение WinAsynchMethod.
2. Откройте форму в режиме конструктора.
3. Выберите пункт меню **View→ToolBox**.
4. Добавьте ЭУ **HelpProvider** на форму.
5. Выделите поле **txbA** для отображения ее свойств.
6. Для свойства **HelpString on helpProvider1** задайте значение **For input integer A.**

7. Постройте и запустите приложение.

8. Переместитесь по форме, используя клавишу **Tab**, до тех пор, пока поле **txbA** не окажется в фокусе.

9. Нажмите на клавишу **F1** для отображения контекстной справки для поля **txbA.**

Простые формы обычно в своем заголовке имеют кнопку с вопросительным знаком, при нажатии на которую курсор меняет свой вид на изображение с вопросом. При щелчке на выбранном элементе управления появляется его краткое описание (подсказка).

Создайте подобную функциональность на форме проекта WinAsynchMethod:

1. Добавьте к имеющимся свойствам формы следующие свойства:

Свойство	**Значение**
MaximizeBox	False
MinimizeBox	False
HelpButton	True
FormBorderStyle	FixedDialog

2. Для полей ввода **txbA**, **txbB** и двух кнопок в свойстве **ShowHelp on helpProvider1** каждого из этих элементов установите значение **True**.

3. Текст, введенный в поле свойства **HelpString on helpProvider1**, будет появляться в качестве подсказки для конкретного элемента. Установите следующие значения этого свойства для каждого элемента:

txbA	For input integer A
txbB	For input integer B
btnRun	Sum
btnWork	Start work

4. Постройте и запустите приложение.

5. Для активации контекстной справки нажмите на кнопку “?”, расположенную в правом верхнем углу приложения.

6. Нажмите на любую кнопку, появится маленькое окошко, объясняющее, что происходит при ее нажатии.

Упражнение 3. Использование справочного файла

1. Откройте Windows-приложение WinAsynchMethod.
2. Откройте форму в режиме конструктора.
3. Выберите пункт меню **View→ToolBox**.
4. Добавьте ЭУ **HelpProvider** на форму (если он не был добавлен ранее).

5. В папке с решением создайте файл справки, например, документ Microsoft Word. Текст укажите произвольный.

6. Для элемента **helpProvider1** в свойстве **HelpNamespace** укажите путь к файлу справки.

7. Реализуйте возможность вызова файла справки созданием либо команды меню, либо кнопки (и команда меню и кнопка может называться, например, **help**).

8. Создайте обработчик события выбора файла справки. В теле обработчика укажите следующую строку:

```
Help.ShowHelp(this,helpProvider1.HelpNamespace);
```

9. Постройте и запустите приложение.

10. Выберите команду вызова справки. Проверьте, что открылся требуемый файл.

Упражнение 4. Добавление всплывающих подсказок

1. Откройте Windows-приложение WinAsynchMethod в режиме конструктора.

2. Выберите пункт меню **View→ToolBox**.

3. Добавьте на форму элемент управления **ToolTip**.

4. В окне **Properties** расположенных на форме элементов и в самой форме появилось свойство **ToolTip on toolTip1**. Установите следующие значения этого свойства для каждого из элементов:

txbA	For input integer A
txbB	For input integer B
btnRun	Sum
btnWork	Start work

5. Постройте и запустите приложение. Проверьте, что при наведении курсора на элемент управления появляется его подсказка.

Локализация приложений

При распространении приложения часто бывает необходимо обеспечивать пользователям возможность работать в своей языковой среде. В связи с этим, при разработке приложений приходится задумываться о переводе пользовательского интерфейса на другие языки. На практике используется два способа решения данной проблемы, первый: создается локальная версия целиком на одном языке и второй: программы содержат многоязычный интерфейс, позволяющий менять оформление приложения непосредственно в ходе работы.

Упражнение 5. Автоматический выбор языка при запуске приложения

В ходе установки операционной системы Windows при определении региональных параметров пользователю предлагается выбрать язык стандартов и форматов. Выбранное значение доступно для изменения в дальнейшем — меню “Пуск” | “Панель управления” | “Язык и региональные параметры” | вкладка “Региональные параметры”.

В этом упражнении вы создадите приложение, которое автоматически будет определять установленный язык стандартов и выводить соответствующий пользовательский интерфейс.

1. Создайте новое Windows-приложение и назовите его WinLanguage.
2. Добавьте на форму кнопку и главное меню.
3. Установите следующие значения свойства **Text** для элементов:

   главное меню – **menu**,

   первая команда – **command one**,

   вторая команда – **command two**,

   кнопка – **Close**,

   для самой формы – **Form**.
4. Для кнопки реализуйте обработчик, закрывающий форму:

```
this.Close();
```

5. Для формы установите свойству **Localizable** значение **True**. Это свойство разрешает поддержку многоязычного интерфейса.

6. В свойстве **Language** выберите значение **English (United States)**.

7. Постройте приложение. В итоге получилась версия программы с интерфейсом на английском языке.

8. В свойстве **Language** выберите **Russian (Russia).**

9. Измените свойство **Text** элементов, заменив названия на английском языке соответствующими названиями на русском.

10. Перейдите в код формы и подключите пространство имен **Threading**:

```
using System.Threading;
```

11. В конструкторе формы до `InitializeComponent();` установите культуру пользовательского интерфейса равной текущей культуре:

```
Thread.CurrentThread.CurrentUICulture =
Thread.CurrentThread.CurrentCulture;
```

12. Постройте и запустите приложение. Среда CLR проверяет установленный язык и выводит приложение с интерфейсом на языке, установленном на вкладке "Региональные параметры" в настройках инструмента "Язык и региональные параметры".

13. Закройте приложение. Перейдите в "Панель управления" и установите другой язык (например, "Английский (США)") на вкладке "Региональные

параметры" в настройках инструмента "Язык и региональные параметры". Снова запустите приложение. Теперь интерфейс приложения должен быть на другом языке (например, английском).

14. При локализации приложения среда Visual Studio .NET создает сборку, в которой хранятся все данные о приложении. В окне Solution Explorer нажмите на кнопку (**Show All Files**) для просмотра добавленных файлов. Названия файлов-ресурсов (Form1. en-US и Form1.ru-RU) содержат в себе указание на язык (первая часть – en или ru) и регион (вторая часть US или RU).

Упражнение 6. Локализация приложения

В этом упражнении вы локализуете пользовательский интерфейс приложения и добавите в него локализованные строковые ресурсы.

➢ **Локализация формы Windows-приложения**

1. Откройте стартовый проект UsabilityDemo.sln из папки **install_folder**\Practices\Mod08\Mod08_04\.
2. Откройте файл UsabilityDemo.cs в режиме конструктора.
3. Нажмите на кнопку **Show All Files** в окне **Solution Explorer**.
4. Обратите внимание на значение свойства формы **Localizable**. Оно установлено в **True**, что означает, что форма может быть локализована.
5. Для свойства формы **Language** задайте значение **French(France).**

В данный момент вы видите английскую версию формы, но теперь вы можете преобразовать ее во французскую. Обратите внимание на то, что в окне **Solution Explorer** под файлом UsabilityDemo.cs появилось несколько новых файлов ресурсов (UsabilityDemo.fr.resx и UsabilityDemo.fr-FR.resx). Обратите внимание на то, что данная форма также была локализована для Германии и Японии.

6. Для свойства формы **Text** задайте значение **Démonstration de l'utilisation**.

***Замечание*:** Текст, необходимый для создания французской версии формы можно скопировать из файла UsabilityDemoLocalizedStrings.rtf, расположенного в папке **install_folder**\Practices\Mod08\Mod08_04.

7. Для свойства **Text** меню **Help** и пункта меню **Help** задайте значение **Aide.**
8. Для свойства **Text** кнопки **Choose a Culture** задайте значение **Choisir une langue.**
9. Для свойства **Text** кнопки **Show Date/Time** задайте значение **Afficher la date/l'heure.**
10. Для свойства **Text** кнопки **Show Currency** задайте значение **Afficher la devise.**

11. Для свойства **Text** кнопки **Show a String** задайте значение **Afficher une chaîne.**

12. Для свойства **Text** кнопки **Exit** задайте значение Quitter**.**

➢ **Добавление в приложение файл строковых ресурсов**

1. В окне **Solution Explorer** ПКМ по проекту **UsabilityDemo | Add | Add New Item.**

2. В окне **Add New Item** выберите **Assembly Resource File.**

3. Задайте для файла ресурсов имя **UsabilityDemoText.fr-FR.resx** и нажмите на кнопку **Open**.

4. В строке ресурсов задайте для **Name** значение **SimpleTextString.**

5. В строке ресурсов задайте для **Value** значение **Voici du texte.**

6. Сохраните и закройте файл ресурсов.

➢ **Добавление кода для получения значений строковых ресурсов**

1. Откройте файл с кодом UsabilityDemo.cs.

2. В окне Task List отобразите комментарии TODO. Для этого выберите пункт меню **View | Show Tasks | All.**

3. Добавьте три директивы **using,** для возможности поддержки локализации приложения.

```
using System.Globalization;
using System.Resources;
using System.Threading;
```

4. Найдите в коде второй комментарий TODO. Объявите **private** переменную типа **ResourceManager**.

```
private ResourceManager RM;
```

5. Найдите в коде следующий комментарий TODO. Создайте экземпляр класса **ResourceManager.** Код добавляется в первый конструктор приложения:

```
RM = new ResourceManager("UsabilityDemo.UsabilityDemoText",
Assembly.GetExecutingAssembly());
```

6. Найдите в коде следующий комментарий TODO. Создайте экземпляр класса **ResourceManager.** Код добавляется во второй конструктор приложения:

```
RM = new ResourceManager("UsabilityDemo.UsabilityDemoText",
Assembly.GetExecutingAssembly());
```

7. Найдите в коде следующий комментарий TODO. Добавьте код использования менеджера ресурсов для получения строки текста из файла ресурсов и отображения ее в текстовом поле.

```
OutputTextBox.Text = RM.GetString("SimpleTextString");
```

8. Найдите в коде следующий комментарий TODO. Добавьте код для задания свойствам **Culture** и **UICulture** значений, выбранных пользователем.

```
Thread.CurrentThread.CurrentUICulture = new CultureInfo(ChosenCulture,
false);
```

```
Thread.CurrentThread.CurrentCulture = new CultureInfo(ChosenCulture,
false);
```

9. Сохраните проект.

➢ **Тестирование работы приложения**

1. Постройте и запустите приложение.

2. Обратите внимание на то, что при старте приложения все, что вы можете – это либо выбрать язык, либо завершить работу приложения.

3. Нажмите на кнопку **Choose a Culture**.

4. Выберите один из языков. Если вы не выберите ни один из них, по умолчанию будет использоваться English.

5. После нажатия на кнопку **OK** на форме **Culture Chooser** эта форма исчезает и появляется та версия формы **UsabilityDemo,** которая соответствует выбранным настройкам.

Лабораторная работа 9. Настройка приложений для распространения

Упражнение 1. Использование строго именованной сборки

В этом упражнении вы сгенерируете цифровую подпись и создадите ссылку на данный файл из библиотеки CalculatorEngine. Далее вы создадите приложение WindowsCalculator, ссылающееся на строго именованную сборку. С помощью программы ILDASM вы просмотрите метаданные для CalculatorEngine.dll и WindowsCalculator.exe.

➢ **Создание строго именованной сборки**

1. Откройте окно командной строки, выбрав **Start |All Programs | Microsoft Visual Studio .NET | Visual Studio .NET Tools | Visual Studio .NET Command Prompt**.

2. В окне командной строки перейдите в директорию **install_folder**\Practices\Mod09\Mod09_01\Starter\CalculatorEngine.

3. Наберите команду **sn –k CalcKey.snk.** Она сгенерирует файл с цифровой подписью, которую вы сможете использовать при создании строго именованной сборки.

4. В Visual Studio .NET откройте проект CalculatorEngine.sln из папки **install_folder**\Practices\Mod09\Mod09_01\Starter\CalculatorEngine.

5. Просмотрите комментарии TODO в окне Task List. Для этого выберите пункт меню **View | Show Tasks | All.**

6. Откройте файл AssemblyInfo.cs

7. Перейдите к фрагменту кода, связанному с первым комментарием TODO. Измените номер версии с 2.0.1.1 на **3.0.1.1.**

```
[assembly: AssemblyVersion("3.0.1.1")]
```

8. Перейдите к следующему комментарию TODO. Внизу файла добавьте новый атрибут для ссылки на файл, содержащий цифровую подпись для создания строго имени.

```
[assembly: AssemblyKeyFile("CalcKey.snk")]
```

9. Откройте файл с кодом Calculator.cs.

10. Перейдите к комментарию TODO. Измените информацию о версии с v2.0.1.1 на **v3.0.1.1.**

```
private static string versionInfo = "Calculator v3.0.1.1";
```

11. Перестройте сборку и закройте Visual Studio .NET.

12. В окне командной строки Visual Studio .NET перейдите в директорию **install_folder**\Practices\Mod09\Mod09_01\Starter\CalculatorEngine\bin\Debug.

13. В окне командной строки запустите ILDASM для CalculatorEngine.dll.

14. Раскройте узел MANIFEST.

15. Обратите внимание, что под **.assembly CalculatorEngine** указан **.publickey.** Это указывает на то, что CalculatorEngine является строго именованной сборкой.

➢ **Создание приложения, ссылающееся на строго именованную сборку**

1. Откройте проект WindowsCalculator.sln из папки **install_folder**\Practices\Mod09\Mod09_01\Starter. Перестройте проект.

2. Запустите приложение и обратите внимание на то, что номер версии 3.0.1.1. Это номер версии CalculatorEngine.dll.

3. В окне командной строки Visual Studio .NET перейдите в директорию **install_folder**\Practices\Mod09\Mod09_01\Starter\bin\Debug.

4. Запустите ILDASM для WindowsCalculator.exe, набрав следующую команду:

```
ildasm WindowsCalculator.exe
```

5. Раскройте узел MANIFEST. Обратите внимание, что для **.assembly extern CalculatorEngine** указаны номер версии 3:0:1:1 и **.publickeytoken.** Это указывает на то, что приложение ссылается на внешнюю строго именованную сборку.

Упражнение 2. Работа с глобальным кэшем сборок

В этом упражнении вы добавите в глобальный кэш сборок (GAC) сборку CalculatorEngine. Затем вы построите приложение WindowsCalculator. Перед запуском приложения Вы удалите локальные копии CalculatorEngine.dll. После чего Вы убедитесь в успешном запуске приложения WindowsCalculator, т.к. ему удастся обнаружить нужную версию CalculatorEngine в глобальном кэше сборок.

Внимание: В процессе разработки вы можете использовать утилиту GACUtil.exe для добавления сборки в глобальный кэш сборок на этапе

тестирования. Это программа используется для удобства и не должна использоваться при развертывании production-версии приложения. В этом случае необходимо добавлять сборки в глобальный кэш сборок с помощью Windows Installer или .NET Framework configuration tool Mscorcfg.msc.

- **Построение новой версии сборки CalculatorEngine**

1. Откройте проект CalculatorEngine.dll из папки **install_folder**\Practices\Mod09\Mod09_02\Starter\ CalculatorEngine.

2. Просмотрите комментарии TODO в окне Task List. Для этого выберите пункт меню **View | Show Tasks | All.**

3. Откройте файл AssemblyInfo.cs

4. Перейдите к фрагменту кода, связанному с первым комментарием TODO. Измените номер версии с 3.0.1.1 на **4.0.1.1.**

```
[assembly: AssemblyVersion("4.0.1.1")]
```

5. Откройте файл с кодом Calculator.cs.

6. Перейдите к комментарию TODO. Измените информацию о версии с v3.0.1.1 на **v4.0.1.1.**

```
private static string versionInfo = "Calculator v4.0.1.1";
```

7. Перестройте сборку и закройте Visual Studio .NET.

- **Добавление сборки в глобальный кэш сборок, используя утилиту GACUtils**

1. Откройте окно командной строки, выбрав **Start | All Programs | Microsoft Visual Studio.NET | Visual Studio .NET Tools | Visual Studio .NET Command Prompt**.

2. В окне командной строки перейдите в директорию **install_folder**\Practices\Mod09\Mod09_02\Starter\CalculatorEngine\bin\Debug.

3. Добавьте сборку CalculatorEngine в глобальный кэш сборок с помощью следующей команды:

```
gacutil –i CalculatorEngine.dll
```

4. Вы должны получить сообщение **Assembly successfully added to the cache.**

5. Просмотрите содержимое глобального кэша сборок с помощью следующей команды:

```
gacutil –l
```

6. Найдите в списке CalculatorEngine. Запись должна отражать следующую информацию: “CalculatorEngine, Version=4.0.1.1, Culture-neutral, PublicKeyToken=***, Custom=null”.

- **Построение приложения, ссылающееся на строго именованную сборку, располагающуюся в глобальном кэше сборок**

1. Откройте проект WindowsCalculator.dll из папки **install_folder**\Practices\Mod09\Mod09_02\Starter.

2. В окне Solution Explorer раскройте узел References.

3. ПКМ **References | Add Reference.**

4. В ОД **Add Reference** нажмите на кнопку **Browse** и перейдите папку **install_folder**\Practices\Mod09\Mod09_02\Starter\CalculatorEngine\bin\Debug, где располагается CalculatorEngine.dll. Нажмите на кнопку **Open**, а затем на кнопку **OK**.

5. Откройте файл с кодом CalcUI.cs.

6. Просмотрите комментарии TODO в окне Task List. Для этого выберите пункт меню **View | Show Tasks | All.**

7. Перейдите к фрагменту кода, связанному с комментарием TODO. Добавьте директиву **using** для доступа к Calculator.

```
using Calculator;
```

8. Постройте приложение WindowsCalculator.

9. В окне Windows Explorer переименуйте локальные копии **CalculatorEngine.dll** в **OLD_Calculator.dll.** Это относится к **install_folder**\Practices\Mod09\Mod09_02\Starter\bin\Debug, **install_folder**\Practices\Mod09\Mod09_02\Starter\CalculatorEngine\bin\Debug и **install_folder**\Practices\Mod09\Mod09_02\Starter\CalculatorEngine\obj\Debug.

10. В окне Windows Explorer перейдите в папку **install_folder**\Practices\Mod09\Mod09_02\Starter\bin\Debug.

11. Запустите приложение WindowsCalculator.exe. Обратите внимание на то, что приложение WindowsCalculator успешно запустится, несмотря на то, что локальная копия сборки CalculatorEngine.dll не найдена. Это происходит потому, что среда выполнения находит нужную сборку в глобальном хранилище сборок и загружает ее.

Упражнение 3. Создание и использование файлов конфигурации приложения

В этом упражнении вы добавите в глобальный кэш сборок новую версию сборки **CalculatorEngine.** Далее Вы построите файл конфигурации приложения для приложения WindowsCalculator, заставляющий приложение ссылаться на более новую версию **CalculatorEngine.** Вам не придется перестраивать приложение WindowsCalculator для того, чтобы оно использовало более новую версию **CalculatorEngine.**

➢ **Добавление новой версии сборки CalculatorEngine в глобальный кэш сборок**

1. Откройте окно командной строки.

2. В окне командной строки перейдите в директорию **install_folder**\Practices\Mod09\Mod09_03\Starter.

3. Удалите CalculatorEngine.dll. Это версия 4.0.1.1, на которую в данный момент ссылается WindowsCalculator.

4. Скопируйте **CalculatorEngine.dll.v5011** в **CalculatorEngine.dll.**

5. Добавьте сборку **CalculatorEngine** в глобальный кэш сборок, используя следующую команду:

```
gacutil -i CalculatorEngine.dll
```

6. Вы должны получить сообщение **Assembly successfully added to the cache.**

7. Просмотрите содержимое глобального кэша сборок с помощью следующей команды:

```
gacutil -l
```

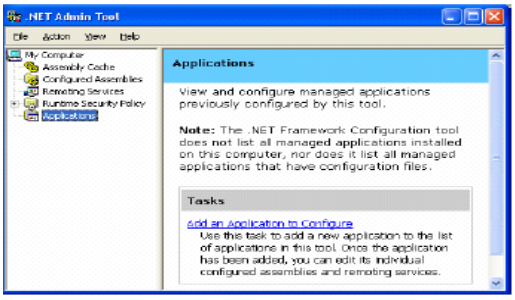
8. Найдите в списке **CalculatorEngine**. В списке должно быть две версии **CalculatorEngine:** версия 4.0.1.1 и версия 5.0.1.1.

➢ **Создание файла конфигурации приложения для приложения WindowsCalculator**

1. Запустите .NET Framework configuration tool (**Mscorcfg.msc**).

2. Раскройте узел **Applications**.

3. Выберите **Add an Application to Configure.**



4. В ОД **Configure an Application** выберите **Other** для указания местонахождения сборки.

5. Перейдите в папку **install_folder**\Practices\Mod09\Mod09_03\Starter, выберите **WindowsCalculator.exe** и нажмите **Open.**

6. В Mscorcfg.msc под Applications раскройте узел приложения WindowsCalculator.exe и выберите **Configured Assemblies**.

7. На правой панели щелкните по ссылке **Configure an Assembly**.

8. В ОД **Configure an Assembly** выберите **Choose an assembly from the assembly cache**.

9. Нажмите на кнопку **Choose Assembly**.

10. В ОД **Choose Assembly from Assembly Cache,** найдите сборку CalculatorEngine версии 5.0.1.1. Выберите CalculatorEngine и нажмите на кнопку **Select**.

11. Нажмите на кнопку **Finish.**

12. Появится ОД **CalculatorEngine Properties**.

13. Выберите вкладку **Binding Policy.**

14. В столбце **Requested Version** наберите **4.0.1.1**

15. В столбце **New Version** наберите **5.0.1.1**

16. Нажмите на кнопку **Apply**, а затем на кнопку **OK**.

17. С помощью программы Notepad откройте файл **install_folder**\Practices\Mod09\Mod09_03\Starter\WindowsCalculator.exe.config и посмотрите его содержимое. Обратите внимание на тэг **<bindingRedirect>.** Закройте Notepad.

18. Запустите приложение **WindowsCalculator.exe.** Убедитесь в том, что оно использует новую версию сборки **CalculatorEngine.** Номер версии будет 5.0.1.1.

Упражнение 4. Публикация приложения с помощью ClickOnce в сетевой папке

В этом упражнении вы опубликуете приложение в сетевой папке, используя ClickOnce, настроите место публикации и загрузки, а затем опубликуете приложение и загрузите его, используя ClickOnce.

1. В корневом каталоге создайте папку с именем C:\ClickOnce.

2. Откройте приложение, которое хотите опубликовать.

3. В Solution Explorer щелкните проект правой кнопкой мыши и затем щелкните команду **Publish...(Опубликовать**). Откроется мастер публикации.

4. В окне **Publish Location** укажите место публикации C:\ClickOnce, нажмите кнопку **Next (Далее)**.

5. В следующем окне установите способ установки приложения пользователями, выбрав вариант из общей папки и указав значение в виде ***computername*****ClickOnce**, где ***computername*** является именем вашего компьютера. Нажмите кнопку **Next (Далее)**.

6. Выберите вариант доступности в автономном режиме (оставьте по умолчанию) Нажмите кнопку **Next (Далее)**.

7. Прочтите информацию об установке и нажмите **Finish (Готово**), чтобы опубликовать ваше приложение в сетевой папке. ClickOnce проверяет требования приложения и публикует его, после чего открывается папка публикации.

8. Откройте папку C:\ClickOnce и дважды щелкните **Setup**. Вы можете увидеть предупреждение о безопасности. Если это так, щелкните **Установить**. Приложение устанавливается и открывается на вашем компьютере. Обратите внимание, что к вашему меню **Пуск** добавляется новая группа.

8. В меню **Пуск** откройте **Панель управления** и выберите **Установка и удаление программ**. Щелкните значок своего приложения и выберите **Заменить/удалить**. Следуйте экранным инструкциям для удаления приложения с вашего компьютера.

Итоговое контрольное задание

Требуется:

Изучить приложение, предлагаемое в данной папке “Calculator_starter”.

На оценку «хорошо»

1. Изменить приложение Calculator_starter (можно в качестве стартового приложения взять пример из последней лабораторной работы) в соответствии со следующими требованиями:
    a. Необходимо сделать так, чтобы пользователь не мог изменять размер формы приложения.
    b. Запретить разворачивать приложение на весь экран.
    c. При запуске форма должна появляться в центре экрана.
    d. При работе приложения форма должна располагаться всегда поверх остальных окон.
    e. Установить вид курсора в виде руки (Hand).
    f. Запретить вводить с клавиатуры любые символы в окно ввода операндов.
    g. Изменить цвет формы (по своему усмотрению).
    h. Добавить меню (содержание и наполняемость продумать самостоятельно).
2. Добавить реализацию дополнительных операций:
    a. Возведение в степень
    b. Извлечение квадратного корня
    c. Расчет обратного значения ($^1/_x$)
    d. Определение квадрата числа
3. Создать для кнопок операций всплывающие подсказки (ЭУ ToolTip).
4. Новую функциональность реализовать как дополнительный вид калькулятора, например, *инженерный*, который раскрывается по выбору команды меню (как у стандартных калькуляторов Windows).

На оценку «отлично»

1. Добавить реализацию двух операций:
    a. Расчет факториала числа (целое)
    b. Извлечение кубического корня
2. Добавить реализацию решения квадратного уравнения (см. проект DemoUr2): коэффициенты уравнения вводятся в **окне диалога**, вызываемого **отдельной кнопкой** калькулятора. Результат решения выводится в текстовое поле в виде строки с указанием значением корней уравнения.

На оценку «отлично+»

Добавить на главную форму второе текстовое поле, в которое выводится значение факториала, рассчитанного в параллельном потоке. Сымитировать длительную операцию.

Литература

1. **Джеффри Рихтер** CLR via C#. Программирование на платформе Microsoft.NET Framework 4.5 на языке C#. – СПб.: Питер, 2016. – 896 с.
2. **Джон Скит C#** для профессионалов. Тонкости программирования. – М.:Вильямс, 2014. – 408 с.
3. **Фленов М**. Библия C#/3-е издание. – СПб.: БХВ-Петербург, 2016. – 544 с.
4. **Троелсен Э., Джепикс Ф**. Язык программирования C# 6.0 и платформа .NET 4.6. – М.:Вильямс, 2016. – 1440 с.
5. **Джозеф Албахари, Бен Албахари.** C# 6.0. Справочник. Полное описание языка– М.:Вильямс, 2016. – 1040 с.